MERMAID

CHEAT SHEETS

A Complete Reference for Diagrams as Code

Flowchart · Sequence · ER · State · Class · Gantt · Timeline · Mindmap · Git Graph · Themes & Styling · Swimlanes · Charts · C4 · Architecture · Kanban · Wardley

46 REFERENCE SHEETS

CONTENTS

INTRODUCTION
01 Introduction & Where Mermaid Runs
02 Diagram Anatomy & Common Syntax
03 Rendering & Tooling
04 Choosing a Diagram Type

FLOWCHART
05 Flowchart — Nodes, Shapes & Direction
06 Flowchart — Links, Subgraphs & Interaction

SEQUENCE DIAGRAM
07 Sequence — Participants & Messages
08 Sequence — Activations, Notes & Control Blocks

ENTITY RELATIONSHIP
09 ER — Entities & Attributes
10 ER — Relationships & Cardinality

STATE DIAGRAM
11 State — States & Transitions
12 State — Composite, Choice & Concurrency

CLASS DIAGRAM
13 Class — Classes & Members
14 Class — Relationships & Cardinality

GANTT & TIMELINE
15 Gantt — Tasks, Sections & Dates
16 Gantt Scheduling & Timeline

MINDMAP
17 Mindmap — Structure & Shapes
18 Mindmap — Icons, Classes & Layout

GIT GRAPH
19 Git Graph — Commits & Branches
20 Git Graph — Merges, Cherry-Picks & Options

STYLING DIAGRAMS
21 Themes, Looks & Configuration
22 Styling Elements & Theme Variables

PROCESS & BEHAVIOR
23 Swimlanes
24 User Journey
25 Use Case Diagram
26 Requirement Diagram
27 ZenUML
28 Event Modeling

CHARTS & DATA
29 Pie Chart
30 Quadrant Chart
31 XY Chart
32 Sankey
33 Radar
34 Treemap
35 Venn

ARCHITECTURE & STRUCTURE
36 C4 Diagram
37 Architecture
38 Block Diagram
39 Packet
40 Tree View

PLANNING & STRATEGY
41 Kanban
42 Ishikawa (Fishbone)
43 Wardley Map
44 Cynefin

QUICK REFERENCE
45 Quick Reference — Core Syntax
46 Quick Reference — Types & Config

MMD

Table of Contents

46 sheets · syntax · examples · notes

INTRODUCTION 01 – 04

- 01 Introduction & Where …
- 02 Diagram Anatomy & C…
- 03 Rendering & Tooling
- 04 Choosing a Diagram Ty…

FLOWCHART 05 – 06

- 05 Flowchart — Nodes, Sh…
- 06 Flowchart — Links, Sub…

SEQUENCE DIAGRAM 07 – 08

- 07 Sequence — Participa…
- 08 Sequence — Activatio…

ENTITY RELATIONSHIP 09 – 10

- 09 ER — Entities & Attribu…
- 10 ER — Relationships & …

STATE DIAGRAM 11 – 12

- 11 State — States & Trans…
- 12 State — Composite, C…

CLASS DIAGRAM 13 – 14

- 13 Class — Classes & Me…
- 14 Class — Relationships …

GANTT & TIMELINE 15 – 16

- 15 Gantt — Tasks, Section…
- 16 Gantt Scheduling & Ti…

MINDMAP 17 – 18

- 17 Mindmap — Structure …
- 18 Mindmap — Icons, Cla…

GIT GRAPH 19 – 20

- 19 Git Graph — Commits …
- 20 Git Graph — Merges, C…

STYLING DIAGRAMS 21 – 22

- 21 Themes, Looks & Confi…
- 22 Styling Elements & Th…

PROCESS & BEHAVIOR 23 – 28

- 23 Swimlanes
- 24 User Journey
- 25 Use Case Diagram
- 26 Requirement Diagram
- 27 ZenUML
- 28 Event Modeling

CHARTS & DATA 29 – 35

- 29 Pie Chart
- 30 Quadrant Chart
- 31 XY Chart
- 32 Sankey
- 33 Radar
- 34 Treemap
- 35 Venn

ARCHITECTURE & STRUCTURE 36 – 40

- 36 C4 Diagram
- 37 Architecture
- 38 Block Diagram
- 39 Packet
- 40 Tree View

PLANNING & STRATEGY 41 – 44

- 41 Kanban
- 42 Ishikawa (Fishbone)
- 43 Wardley Map
- 44 Cynefin

QUICK REFERENCE 45 – 46

- 45 Quick Reference — Cor…
- 46 Quick Reference — Ty…

Introduction & Where Mermaid Runs

WHAT IS MERMAID?

Mermaid is a JavaScript library that turns plain-text definitions into diagrams. You write a few lines of text, and Mermaid parses them and draws an **SVG**. Knut Sveidqvist started it in 2014. Because the source is text, diagrams live next to your code, show up in diffs and code review, and never go stale in a separate drawing tool. This set targets **Mermaid 12.x** (12.2.0) and covers all 31 diagram types in the official docs.

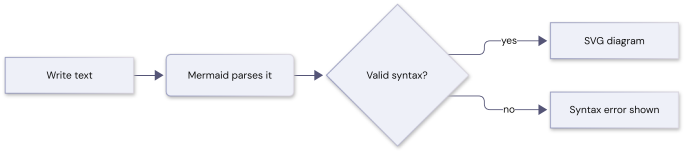
GOOD TO KNOW

- In Markdown, put the diagram in a ```` ```mermaid ```` code fence
- Standalone source files use `.mmd` or `.mermaid`
- Types marked **beta** need the suffix on line 1, e.g. `venn-beta`
- A diagram that is just `info` prints the Mermaid version in use
- Renders on these sheets use a light print theme, not Mermaid's v12 defaults (sheet 21)

HELLO, MERMAID

```
flowchart LR
  A[Write text] --> B(Mermaid parses it)
  B --> C{Valid syntax?}
  C -->|yes| D[SVG diagram]
  C -->|no| E[Syntax error shown]
```

RENDERED OUTPUT



WHERE MERMAID RENDERS

PLATFORM	HOW
GitHub	```` ```mermaid ```` fence in `.md`, issues, PRs, wikis
GitLab	```` ```mermaid ```` fence in Markdown
Azure DevOps	`::: mermaid` … `:::` block in wikis
Obsidian	Built in: ```` ```mermaid ```` fence
Notion	Code block, language set to Mermaid
VS Code	Markdown preview needs a Mermaid extension
JetBrains IDEs	Markdown preview renders fences
Docs sites	Docusaurus, MkDocs, VitePress (sheet 03)

TOOLS

TOOL	USE
mermaid.live	Official live editor: preview, share link, export PNG/SVG
mermaid (npm)	The library: `npm i mermaid` or the jsDelivr CDN
@mermaid-js/mermaid-cli	`mmdc` command: render files to SVG / PNG / PDF
@mermaid-js/tiny	Half-size build: no mindmap, architecture, KaTeX or ELK
Mermaid Chart	Hosted editor and VS Code extension from the Mermaid team

VERSIONS TO KNOW

VERSION	WHAT CHANGED
v10 (2023)	Front matter `config:` replaces `%%{init}%%`; `mermaid.run()`
v11 (2024)	New flowchart shapes `@{ shape: }`, architecture, packet, more
v12 (Sep 2026)	ELK bundled; `redux-color` theme and `neo` look by default for 10 types
12.2.0	Current release when this set was written (Oct 2026)

⚠ COMMON MISTAKES

- Labeling the fence ```` ```mmd ```` on GitHub. Only the language name `mermaid` is rendered; anything else shows as plain code.
- Assuming every platform runs the newest Mermaid. GitHub, GitLab and Obsidian bundle their own version, so beta types and new syntax may fail there.
- Starting with a capitalized or misspelled declaration (`Flowchart` , `sequencediagram`). The first line must match the keyword exactly.

✓ TIPS & BEST PRACTICES

- Prototype in **mermaid.live**, then paste the text into your doc. The share link keeps the diagram in the URL.
- Render an `info` diagram on a platform to see which Mermaid version it runs before using newer syntax.
- Keep large diagrams in `.mmd` files and render them in CI with `mmdc` , so broken syntax fails the build.

QUICK SUMMARY

- `text → SVG` — Mermaid draws diagrams from plain text
- ```` ```mermaid ```` — the Markdown fence most platforms render
- `-beta` — newer diagram types still need the suffix
- `mermaid.live` / `mmdc` — edit online / render from the command line

Sheet 02

Diagram Anatomy & Common Syntax

declaration line · -beta · %% comments · --- front matter · accTitle / accDescr · #quot; · markdown strings

THE SHAPE OF EVERY DIAGRAM

Every Mermaid diagram has the same skeleton: optional **front matter** between --- lines (title and config), a **declaration line** naming the diagram type, then **statements**, usually one per line. %% starts a comment. `accTitle` and `accDescr` add accessible text for screen readers. Labels that contain symbols go in double quotes, and characters that would break the syntax can be written as **entity codes**.

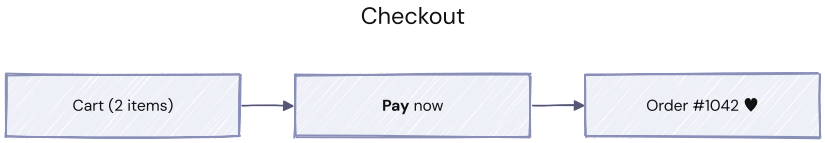
RULES

- Front matter must start on line 1 — nothing before the first `---`
- The declaration is the first line after front matter (comments aside)
- Keywords are case-sensitive: `sequenceDiagram`, `gitGraph`, `erDiagram`
- `%%` comments go on their own line
- `%%{init: … }%%` directives are deprecated since v10.5 — use front matter `config:`

FRONT MATTER, ACCESSIBILITY, COMMENTS & LABELS

```
---
title: Checkout
config:
  look: handDrawn
---
flowchart LR
  accTitle: Checkout flow
  accDescr: Cart to payment to confirmation
  %% comments are ignored by the parser
  cart["Cart (2 items)"] --> pay["`**Pay** now`"]
  pay --> done["Order #35;1042 #9829;"]
```

RENDERED OUTPUT



SKELETON PIECES

SYNTAX	PURPOSE
`---`	Opens and closes the YAML front matter
`title: Text`	Title drawn above the diagram
`config:`	Per-diagram settings: `theme`, `look`, `layout`, …
`flowchart LR`	Declaration: diagram type (+ options)
`%% text`	Comment, ignored when rendering
`accTitle: Text`	Accessible title (<title> in the SVG)
`accDescr: Text`	Accessible description, one line
`accDescr { … }`	Accessible description over several lines

LABELS & ESCAPING

SYNTAX	RESULT
`id[Text]`	Plain label
`id["A (b) & c"]`	Quotes allow ( ) [ ] { } & in a label
`#quot;`	Double quote inside a quoted label
`#35;`	# written as its decimal code
`#9829;`	Any character by code: ♥
`"`**b** _i_`"`	Markdown string: bold, italic, auto-wrap
`Line 1 Line 2`	Line break inside a label

GOTCHAS

PITFALL	FIX
`end`	Lowercase end as a node text breaks flowcharts: write End or quote it
`A---oB`	A node name starting with o / x after a link becomes a circle / cross edge: add a space
`indentation`	Mindmap, treemap, ishikawa and tree view use it as structure
`;`	Optional statement separator; one statement per line is clearer
`tabs vs spaces`	Mix them and indentation-based diagrams nest wrongly

⚠ COMMON MISTAKES

- Putting a comment or blank text above the opening `---`. Front matter is only recognized when `---` is the very first line.
- Pasting from a word processor: curly quotes “ ” are not string delimiters, so labels with symbols fail to parse.
- Writing `%%{init: …}%%` in new diagrams. It still works for now, but it is deprecated; front matter `config:` is the supported form.
- Typing `#` directly in some labels. Use `#35;` or quote the whole label.

✓ TIPS & BEST PRACTICES

- Always add `accTitle` and `accDescr` to diagrams in public docs; they become the SVG's `<title>` and `<desc>`.
- Quote any label with punctuation by default. It costs two characters and avoids most parse errors.
- Use markdown strings (backticks inside quotes) for long labels: they wrap automatically.

QUICK SUMMARY

- `---` — front matter: `title` and `config` for one diagram
- `declaration` — first statement names the diagram type
- `%%` — comment line
- `accTitle` / `accDescr` — accessible title and description
- `"…"` / `#code;` — quote labels, escape characters by code

Rendering & Tooling

GETTING A DIAGRAM ON SCREEN

On a web page, Mermaid finds elements with `class="mermaid"`, reads their text and replaces it with an SVG. Load it as an **ES module** from a CDN or npm, call `mermaid.initialize()` once with site-wide options, and let `startOnLoad` render the page, or call `mermaid.run()` / `mermaid.render()` yourself for content that arrives later. Outside the browser, the **mmdc** command line tool renders `.mmd` files to SVG, PNG or PDF.

NOTES

- Pin a version in CDN URLs (`mermaid@12.2.0`) so diagrams don't change under you
- `mermaid.init()` is deprecated since v10 — use `mermaid.run()`
- Render after fonts load, or labels may be measured with the wrong font
- `securityLevel: 'strict'` (default) blocks click scripts and HTML in labels
- The tiny build omits mindmap, architecture, KaTeX and ELK

MERMAID ON A WEB PAGE

```
<pre class="mermaid">
flowchart LR
  A[Request] --> B[Response]
</pre>

<script type="module">
  import mermaid from
    "https://cdn.jsdelivr.net/npm/mermaid@12.2.0/dist/mermaid.esm.min.mjs";
  mermaid.initialize({ startOnLoad: true, theme: "neutral" });

  // later, for content added after load:
  await mermaid.run({ querySelector: ".late-diagram" });
  const { svg } = await mermaid.render("d1", "pie\n \"A\" : 1");
</script>
```

CODE EXPLAINED

- → `class="mermaid"` — the element's text is the diagram source
- → `import mermaid` — ESM build from jsDelivr, version pinned
- → `initialize()` — site-wide config, call once
- → `startOnLoad` — render every `.mermaid` element on page load
- → `run()` — render elements matched by `querySelector` / `nodes`
- → `render(id, text)` — returns `{ svg, bindFunctions }` for you to insert

JAVASCRIPT API

CALL	DOES
`initialize(config)`	Set site-wide config (theme, look, securityLevel, …)
`run(options)`	Render matching elements; `suppressErrors: true` keeps going
`render(id, text)`	Render one definition → `{ svg, bindFunctions }`
`parse(text, opts)`	Validate only → `{ diagramType }`, or throws
`registerIconPacks([…])`	Add Iconify packs for `icon` / architecture
`registerExternalDiagrams`	Add plugin diagrams such as ZenUML

MMDC (MERMAID-CLI)

COMMAND	RESULT
`npm i -g @mermaid-js/mermaid-cli`	Install the `mmdc` command
`mmdc -i in.mmd -o out.svg`	Render to SVG (format from extension)
`mmdc … -o out.png -s 2`	PNG at 2× scale
`mmdc … -o out.pdf`	PDF output
`-t dark -b transparent`	Theme and background color
`-c config.json`	Mermaid config file
`mmdc -i doc.md -o out.md`	Render every fence; links SVGs into Markdown

DOCS SITES & EDITORS

TOOL	SETUP
Docusaurus	`markdown.mermaid: true` + `@docusaurus/theme-mermaid`
MkDocs Material	`pymdownx.superfences` custom fence `mermaid`
VitePress	`vitepress-plugin-mermaid`
Sphinx	`sphinxcontrib-mermaid`
VS Code	Markdown Preview Mermaid Support extension

⚠ COMMON MISTAKES

- Calling `initialize()` with `startOnLoad: true` and also calling `run()` on the same elements. They render twice, or the second pass sees SVG instead of source.
- HTML-escaping the source wrong: inside `<pre class="mermaid">` write `-->` as is, but `<` and `&` must be `<` and `&` .
- Rendering into a hidden tab or collapsed `<details>` : elements with zero width produce squashed or empty diagrams. Render when visible.

✓ TIPS & BEST PRACTICES

- Use `mermaid.parse(text, { suppressErrors: true })` to validate user input before rendering.
- In CI, run `mmdc` on every `.mmd` file so a syntax error fails the build.
- Give each `render()` call a unique id; reusing one id overwrites the previous temporary element.

QUICK SUMMARY

- `class="mermaid"` + `startOnLoad` — simplest page setup
- `initialize()` — global config, once per page
- `run()` / `render()` — render on demand
- `mmdc -i … -o …` — render files to SVG / PNG / PDF

Choosing a Diagram Type

PICK THE DIAGRAM BY THE QUESTION

Start from the question the reader needs answered, not from the diagram you know best. **Flowcharts** show steps and decisions, **sequence diagrams** show who talks to whom and in what order, **ER** and **class** diagrams show structure, **state** diagrams show lifecycles, and the chart types turn numbers into pictures. The tables below match common goals to the diagram that fits, with the declaration keyword and the sheet that covers it.

RULES OF THUMB

- Steps over time with decisions → flowchart
- Messages between systems → sequence diagram
- Tables and keys → ER diagram; code types → class diagram
- Something that changes status → state diagram
- Dates and durations → Gantt; dated events → timeline
- When two fit, choose the one with fewer lines of source

PROCESS & BEHAVIOR

DECLARATION	GOAL (SHEET)
`flowchart`	Steps, branches, decisions (05)
`swimlane-beta`	Steps split by owner / team (23)
`sequenceDiagram`	Calls between services (07)
`zenuml`	Calls written as code (27)
`stateDiagram-v2`	Status lifecycle (11)
`journey`	User's experience by step (24)
`usecase-beta`	Actors and what they can do (25)
`requirementDiagram`	Requirements traceability (26)
`eventmodeling`	Event-sourced system design (28)
`ishikawa-beta`	Root causes of a problem (42)

STRUCTURE & ARCHITECTURE

DECLARATION	GOAL (SHEET)
`erDiagram`	Database tables and keys (09)
`classDiagram`	Classes, interfaces, types (13)
`mindmap`	Ideas branching from a topic (17)
`gitGraph`	Branches, commits, merges (19)
`C4Context`	System context / containers (36)
`architecture-beta`	Cloud services and links (37)
`block`	Blocks you place by hand (38)
`packet`	Bit fields in a header (39)
`treeView-beta`	Folder / file tree (40)
`wardley-beta`	Value chain vs evolution (43)

PLANS & NUMBERS

DECLARATION	GOAL (SHEET)
`gantt`	Tasks with dates, dependencies (15)
`timeline`	Events grouped by period (16)
`kanban`	Work items by status column (41)
`pie`	Share of a whole (29)
`quadrantChart`	Items on two axes, 4 quadrants (30)
`xychart`	Bars / lines over categories (31)
`sankey`	Flow quantities between stages (32)
`radar-beta`	Scores on several axes (33)
`treemap-beta`	Hierarchy sized by value (34)
`venn-beta`	Overlap between sets (35)
`cynefin-beta`	Which kind of problem is it? (44)

⚠ COMMON MISTAKES

- Forcing everything into a flowchart. Message order between services reads far better as a sequence diagram.
- Using a pie chart for more than about six slices or for values that don't add up to a whole; use `xychart` bars instead.
- Drawing a database schema as a class diagram. `erDiagram` has keys and crow's-foot cardinality built in.

✓ TIPS & BEST PRACTICES

- Sketch the question first ("what happens when…", "what is connected to…") — the verb usually names the diagram type.
- Split one huge diagram into several small ones of the right type rather than one of the wrong type.
- Check platform support before choosing a beta type for docs that render on GitHub or GitLab.

QUICK SUMMARY

- `flowchart` / `sequenceDiagram` — process / interactions
- `erDiagram` / `classDiagram` — data / code structure
- `stateDiagram-v2` — lifecycles
- `gantt` / `timeline` — schedules / history
- `pie` · `xychart` · `sankey` — numbers

Flowchart — Nodes, Shapes & Direction

NODES AND DIRECTION

A `flowchart` draws **nodes** joined by **links**. The declaration sets the direction (TD, LR, …). A node is an **id** plus optional text in brackets, and the bracket style picks the shape: `[ ]` box, `( )` rounded, `{ }` diamond. Since v11.3 you can also name any of 50+ shapes with `id@{ shape: doc, label: "…" }`. In v12, flowcharts default to the `redux-color` theme and `neo` look.

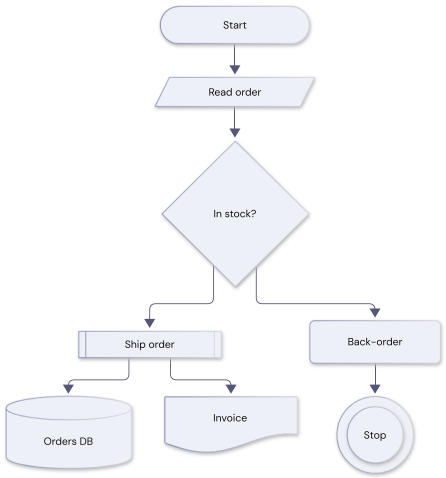
NODE RULES

- `graph` is the older keyword; `flowchart` has more features — use it
- Declare an id once; later mentions reuse the same node
- With no text, the id is the label: `A` draws a box labeled A
- Quote labels with symbols: `A["Save (draft)"]`
- Unicode works in quotes: `A["Größe • Café"]`

ORDER HANDLING

```
flowchart TD
  start([Start]) --> read[/Read order/]
  read --> stock{In stock?}
  stock --> ship[[Ship order]]
  stock --> wait(Back-order)
  ship --> db[(Orders DB)]
  ship --> inv@{ shape: doc, label: "Invoice" }
  wait --> stop(((Stop)))
```

RENDERED OUTPUT



BRACKET SHAPES

SYNTAX	SHAPE
`A[Text]`	Rectangle — process
`A(Text)`	Rounded rectangle
`A([Text])`	Stadium — start / end
`A[[Text]]`	Subroutine
`A[(Text)]`	Cylinder — database
`A((Text))`	Circle
`A(((Text)))`	Double circle
`A>Text]`	Flag / asymmetric
`A{Text}`	Diamond — decision
`A{{Text}}`	Hexagon
`A[/Text/]`	Parallelogram — input
`A[\Text\]`	Parallelogram, leaning left
`A[/Text\]`	Trapezoid
`A[\Text/]`	Trapezoid, inverted

NAMED SHAPES (V11.3+)

SHAPE:	MEANING
`rect`	Process
`diam`	Decision
`doc / docs`	Document / multiple documents
`cyl / lin-cyl`	Database / disk storage
`hex`	Prepare / condition
`sm-circ / fr-circ`	Start / stop
`fork`	Fork or join bar
`delay`	Delay (half-rounded)
`bolt`	Communication link
`brace`	Comment
`cloud / person`	Cloud / person
`text`	Text block, no border

DIRECTION & TEXT

SYNTAX	MEANING
`flowchart TD`	Top → down (same as TB)
`flowchart BT`	Bottom → top
`flowchart LR`	Left → right
`flowchart RL`	Right → left
`A@{ shape: x, label: "…" }`	Named shape with label
`A["`**bold** *it*`"]`	Markdown string, wraps
`A@{ icon: "fa:user" }`	Icon shape (needs icon pack)
`A@{ img: "url" }`	Image shape

⚠ COMMON MISTAKES

- Writing `end` as node text. Lowercase `end` closes a subgraph and breaks the chart; use `End` or `["end"]`.
- Giving one id two different labels. They are one node, and the later text replaces the earlier one.
- Using `@{ shape: … }` on a platform running Mermaid older than 11.3 — the whole diagram fails to parse.

✓ TIPS & BEST PRACTICES

- Use stadium `([ ])` for start and end, and diamond `{ }` only for decisions, so readers can scan the flow.
- Keep ids short (`chk`) and put the words in the label (`chk{Valid?}`); ids are what you type in links.
- Prefer `LR` for pipelines and `TD` for decision trees and hierarchies.

QUICK SUMMARY

- `flowchart TD|LR|BT|RL` — declaration sets direction
- `id[text]` — brackets choose the shape
- `id@{ shape: name }` — 50+ named shapes
- `"…"` — quote labels with symbols or Unicode

Flowchart — Links, Subgraphs & Interaction

LINKS AND GROUPS

Links connect nodes; the characters set the style (`-->` arrow, `-.->` dotted, `==>` thick) and the end (`o`, `x`, `>`). Text goes between pipes `-->|text|`. Chains and `&` let one line make several links. **Subgraphs** group nodes in a titled box and can have their own direction. Since v11 an edge can carry an **id** (`e1@-->`) so you can animate or restyle that one link.

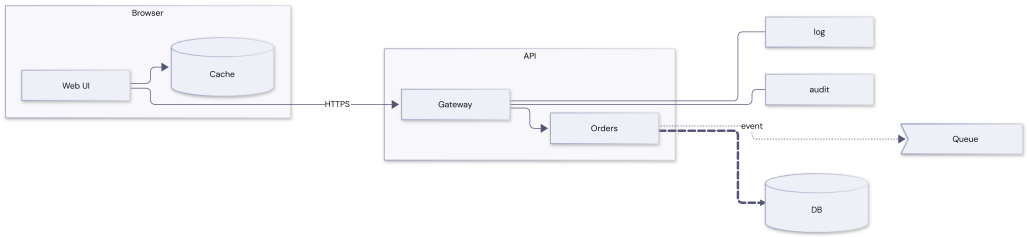
LINK RULES

- Each extra dash makes a link span one more rank: `---->`
- `linkStyle` counts links from 0 in the order they are written
- Links may point at a subgraph id as if it were a node
- `click` only works with `securityLevel: 'loose'`
- Escape commas in style values as `\,`

CHECKOUT SERVICES

```
flowchart LR
  subgraph web [Browser]
    direction TB
    ui[Web UI] --> cache[(Cache)]
  end
  subgraph api [API]
    gw[Gateway] --> svc[Orders]
  end
  ui -->|HTTPS| gw
  svc -.->|event| q>Queue]
  svc e1@==> db[(DB)]
  e1@{ animate: true }
  gw --- log & audit
```

RENDERED OUTPUT



LINK TYPES

SYNTAX	LINK
`A --> B`	Arrow
`A --- B`	Line, no arrowhead
`A -.-> B`	Dotted arrow
`A ==> B`	Thick arrow
`A ~~~ B`	Invisible (layout only)
`A --o B`	Circle end
`A --x B`	Cross end
`A <--> B`	Arrows on both ends
`A ----> B`	Longer: +1 rank per dash
`A -->|text| B`	Label (pipe form)
`A -- text --> B`	Label (inline form)

CHAINS, IDS & STYLES

SYNTAX	EFFECT
`A --> B --> C`	Chain of links
`A & B --> C`	Two links into C
`A --> B & C`	Two links out of A
`A e1@--> B`	Give the edge id `e1`
`e1@{ animate: true }`	Animate that edge
`e1@{ animation: slow }`	Animate, `fast` / `slow`
`e1@{ curve: linear }`	Curve for one edge (v11.10)
`linkStyle 0 stroke:red`	Style link #0
`linkStyle default …`	Style every link

SUBGRAPHS & CLICKS

SYNTAX	EFFECT
`subgraph id [Title]`	Open a titled group
`end`	Close the group
`direction LR`	Direction inside a subgraph
`A --> groupId`	Link to a whole group
`id@{ view: collapsed }`	Draw group as one node (v11.17)
`click A "url" "tip"`	Open a link, with tooltip
`click A href "url" _blank`	Link in a new tab
`click A call fn()`	Run a page function

⚠ COMMON MISTAKES

- Counting `linkStyle` indexes from 1. They start at 0 and include every link, even ones made by `&` and chains.
- Expecting `direction` inside a subgraph to win when its nodes link outside it; external links make the subgraph use the parent direction.
- Starting the target id with `o` or `x` (`A---oven`) — Mermaid reads a circle or cross edge. Add a space: `A--- oven`.

✓ TIPS & BEST PRACTICES

- Use edge ids instead of `linkStyle` numbers: `e1@{ … }` keeps working when you reorder lines.
- Use `~~~` invisible links to nudge layout without drawing anything.
- Name subgraphs with an id and a title (`subgraph api [Public API]`) so you can link to them.

QUICK SUMMARY

- `-->` / `-.->` / `==>` — arrow, dotted, thick
- `-->|text|` — label a link
- `&` — fan links in or out on one line
- `subgraph … end` — titled group with its own direction
- `e1@-->` + `e1@{…}` — style or animate one edge

Sequence — Participants & Messages

sequenceDiagram · participant / actor · as alias · ->> · -->> · create / destroy · autonumber

WHO TALKS TO WHOM

A `sequenceDiagram` shows **participants** across the top and **messages** between them in time order, top to bottom. Participants appear when first used, or in the order you declare them with `participant` / `actor`. Each message line is `From--<<To: text`; the arrow sets solid vs dotted and the arrowhead. Since v11 a participant can take a **type** such as `database` or `queue` to change its symbol.

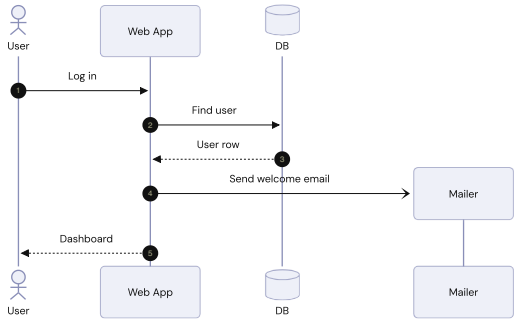
NOTES

- Declare participants first to control their left-to-right order
- `as` gives a short id a longer display name
- Only the **receiver** of a message can be created mid-diagram
- Hex colors don't work in `box` — `#` starts a comment; use names or `rgb()`
- v12 default theme: `redux-color`, look `neo`

LOGIN

```
sequenceDiagram
  autonumber
  actor U as User
  participant W as Web App
  participant DB@{ "type": "database" }
  U->>W: Log in
  W->>DB: Find user
  DB-->>W: User row
  create participant M as Mailer
  W-)M: Send welcome email
  W-->>U: Dashboard
```

RENDERED OUTPUT



PARTICIPANTS

SYNTAX	MEANING
`participant A`	Box (declares order)
`actor A`	Stick figure
`participant A as Alice`	Short id, long label
`A@{ "type": "database" }`	Database symbol
`"type": "boundary"`	Boundary symbol
`"type": "control"`	Control symbol
`"type": "entity"`	Entity symbol
`"type": "collections"`	Stacked collection
`"type": "queue"`	Queue symbol
`create participant B`	B appears at next message
`destroy B`	B's lifeline ends (✕)

MESSAGE ARROWS

ARROW	LINE
`->`	Solid, no head
`-->`	Dotted, no head
`->>`	Solid, arrowhead (call)
`-->>`	Dotted, arrowhead (reply)
`<<->>`	Solid, both ends
`<<-->>`	Dotted, both ends
`-x / --x`	Cross at end (lost / failed)
`-) / --)`	Open head (async)
`A->>()B`	To the lifeline center (v11.12)

GROUPING & NUMBERING

SYNTAX	EFFECT
`box Aqua Frontend`	Group participants (until end)
`box rgb(230,240,255)`	Box with an rgb color
`autonumber`	Number every message
`autonumber 10 5`	Start at 10, step 5 (v11.15)
`A->>B: one two`	Line break in message
`#59;`	Semicolon by code (`;` ends lines)
`%% text`	Comment

⚠ COMMON MISTAKES

- Using `->` when you meant a call. `->` and `-->` draw no arrowhead; use `->>` for requests and `-->>` for replies.
- Writing `box #eef Frontend`. The `#` starts a comment, so the box loses its color and label.
- Trying to `create` the sender of a message. Only the recipient can be created; the sender must already exist.

✓ TIPS & BEST PRACTICES

- Use `actor` for people and `participant` (with a type) for systems so the roles read at a glance.
- Stick to one convention: solid `->>` for requests, dotted `-->>` for responses, `-)` for fire-and-forget.
- Add `autonumber` when the diagram will be discussed in review; people can say "step 4".

QUICK SUMMARY

- `participant` / `actor` — declare and order lifelines
- `A@{ "type": … }` — database, queue, boundary symbols
- `->>` / `-->>` — request / reply
- `create` / `destroy` — lifelines that start or end mid-way
- `autonumber` — number the messages

Sheet 08

Sequence — Activations, Notes & Control Blocks

activate / +- · Note over · loop · alt / else · opt · par / and · critical · break · rect

SHOWING LOGIC IN TIME

Activations draw a thin bar on a lifeline while a participant is busy: use `activate` / `deactivate`, or the shortcut + / – on a message arrow. **Notes** sit beside or across lifelines. **Control blocks** wrap messages in a labeled frame: `loop` for repetition, `alt`/`else` for branches, `opt` for optional steps, `par`/`and` for parallel work, `critical` and `break` for error paths. Every block closes with `end`.

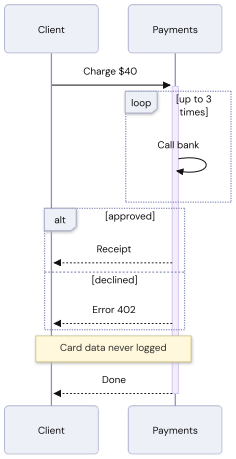
NOTES

- `+` activates the receiver, `-` deactivates the sender
- Activations can stack on the same lifeline
- Blocks nest: an `alt` inside a `loop` is fine
- `rect` takes a color: `rect rgb(240,240,255)` … `end`
- Each block needs its own `end`

PAYMENT WITH RETRY

```
sequenceDiagram
  participant C as Client
  participant P as Payments
  C->>+P: Charge $40
  loop up to 3 times
    P->>P: Call bank
  end
  alt approved
    P-->>C: Receipt
  else declined
    P-->>C: Error 402
  end
  Note over C,P: Card data never logged
  P-->>-C: Done
```

RENDERED OUTPUT



ACTIVATIONS & NOTES

SYNTAX	EFFECT
`activate B`	Start a bar on B
`deactivate B`	End B's bar
`A->>+B: msg`	Send and activate B
`B-->>-A: msg`	Reply and deactivate B
`Note right of A: …`	Note to the right
`Note left of A: …`	Note to the left
`Note over A: …`	Note on one lifeline
`Note over A,B: …`	Note across two lifelines

CONTROL BLOCKS

BLOCK	USE
`loop label`	Repeat
`alt label`	First branch of if / else
`else label`	Next branch (inside `alt`)
`opt label`	Optional step, no else
`par label`	Parallel actions
`and label`	Next parallel branch
`critical label`	Must happen atomically
`option label`	Alternative inside `critical`
`break label`	Stop the flow (exception)
`rect rgb(…)`	Highlight background
`end`	Close any block

USEFUL CONFIG

SEQUENCE: KEY	EFFECT
`mirrorActors: false`	No participant boxes at bottom
`showSequenceNumbers`	Same as `autonumber`
`actorMargin: 50`	Space between lifelines
`messageAlign: left`	Message text alignment
`wrap: true`	Wrap long message text
`hideUnusedParticipants`	Drop unused participants

⚠ COMMON MISTAKES

- Deactivating a participant that isn't active. Mermaid throws an error; every `-` or `deactivate` needs a matching activation.
- Forgetting `end` after `alt` or `loop`. The parser reports an error at the end of the diagram, not where the block opened.
- Putting `else` inside `opt`. `opt` has no branches; use `alt … else … end` instead.

✓ TIPS & BEST PRACTICES

- Use the `+` / `-` shortcuts; they keep activation next to the message that causes it.
- Label blocks with the condition (`alt card declined`), not just the outcome.
- Use `Note over A,B` for protocol rules that apply to a whole exchange.

QUICK SUMMARY

- `activate` / `+` `-` — busy bars on lifelines
- `Note over A,B` — notes beside or across lifelines
- `loop` · `alt/else` · `opt` — repeat, branch, optional
- `par/and` · `critical` · `break` — parallel and error paths
- `end` — closes every block

ER — Entities & Attributes

ENTITIES AND THEIR COLUMNS

An `erDiagram` models data: **entities** (usually tables) and the **relationships** between them. Give an entity a block `{ }` of attributes, one per line as `type name`, optionally followed by keys (`PK`, `FK`, `UK`) and a quoted comment. Mermaid doesn't check types, so write whatever your database uses. Since v12, ER diagrams default to the `redux-color` theme and `neo` look.

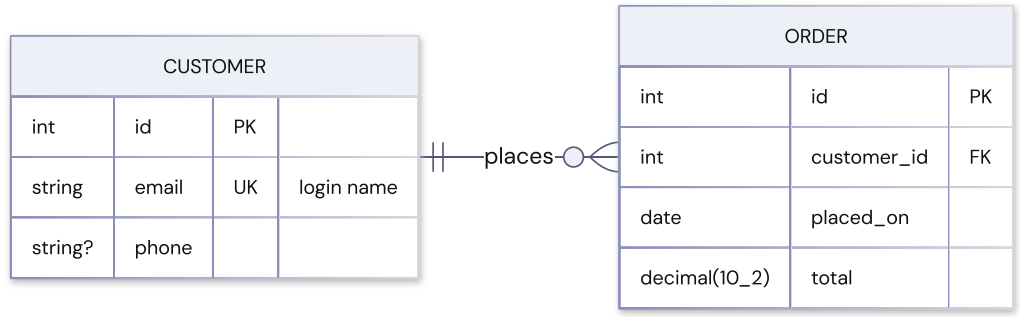
RULES

- Entity names: letters, digits, `-` and `_`; quote others
- Types start with a letter; may hold `( ) [ ]`, e.g. `varchar(255)`
- End a type with `?` for nullable (v11.16): `string? nickname`
- Several keys: separate with commas — `PK, FK`
- Comments can't contain double quotes

SHOP SCHEMA

```
erDiagram
  direction LR
  CUSTOMER {
    int id PK
    string email UK "login name"
    string? phone
  }
  ORDER {
    int id PK
    int customer_id FK
    date placed_on
    decimal(10_2) total
  }
  CUSTOMER ||--o{ ORDER : places
```

RENDERED OUTPUT



ATTRIBUTE SYNTAX

SYNTAX	MEANING
`ENTITY { … }`	Attribute block
`string name`	`type name`
`int id PK`	Primary key
`int user_id FK`	Foreign key
`string email UK`	Unique key
`int id PK, FK`	Several keys
`string sku "stock code"`	Comment
`string? middle`	Nullable type (v11.16)
`string[] tags`	Array-style type
`*id`	Leading * also marks a PK

ENTITIES & LAYOUT

SYNTAX	MEANING
`CUSTOMER`	Entity with no attributes
`p[Person]`	Alias: draw `Person` for id p
`a["Order Line"]`	Alias with spaces
`direction LR`	TB (default), BT, LR, RL
`subgraph Sales … end`	Group entities (v11.17)
`style ORDER fill:#fee`	Style one entity
`classDef / class / :::`	Reusable styles (sheet 22)

COMMON TYPE WORDS

TYPE	TYPICAL USE
`int / bigint`	Ids, counts
`string / varchar(n)`	Text
`decimal(10_2)`	Money (commas not allowed: use _)
`date / datetime`	Dates and timestamps
`boolean`	Flags
`uuid`	Public ids
`json`	Unstructured payload

⚠ COMMON MISTAKES

- Writing `decimal(10,2)`. The comma ends the type; use `decimal(10_2)` or a comment for precision.
- Using spaces in an entity name (`ORDER LINE`). Use `ORDER-LINE`, or an alias: `ol["Order Line"]`.
- Putting the key before the name (`int PK id`). The order is always `type name keys "comment"`.

✓ TIPS & BEST PRACTICES

- Mirror your real schema names so the diagram doubles as documentation; generate it from migrations if you can.
- Show only key columns on big diagrams; full column lists make every box huge.
- Use `direction LR` for wide schemas so boxes line up as a chain.

QUICK SUMMARY

- `erDiagram` — entities and relationships
- `ENTITY { type name }` — attributes, one per line
- `PK` / `FK` / `UK` — keys after the name
- `"comment"` / `type?` — notes and nullable types

ER — Relationships & Cardinality

CROW'S FOOT NOTATION

A relationship line reads `A <left-marker><line><right-marker> B : label`. Each marker has two characters: the **outer** one is the maximum (`|` one, `{/}` many) and the **inner** one is the minimum (`o` zero, `|` one). The line is `--` for an **identifying** relationship (solid) or `..` for **non-identifying** (dashed). The label is required and describes the relationship from left to right.

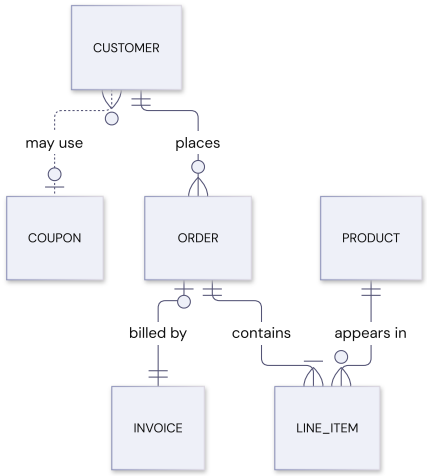
READING A LINE

- `CUSTOMER ||--o{ ORDER` — one customer, zero or more orders
- Outer char = maximum, inner char = minimum
- Left marker describes the left entity's count
- Labels with spaces need quotes: `: "is billed to"`
- Word aliases exist: `1 to zero or more`

ORDERS AND PRODUCTS

```
erDiagram
  CUSTOMER ||--o{ ORDER : places
  ORDER ||--|{ LINE_ITEM : contains
  PRODUCT ||--o{ LINE_ITEM : "appears in"
  CUSTOMER }o..o| COUPON : "may use"
  ORDER |o--|| INVOICE : "billed by"
```

RENDERED OUTPUT



CARDINALITY MARKERS

LEFT / RIGHT	MEANING
`|o o|`	Zero or one
`|| ||`	Exactly one
`}o o{`	Zero or more
`}| |{`	One or more

LINES & PATTERNS

SYNTAX	MEANING
`--`	Identifying (solid line)
`..`	Non-identifying (dashed)
`A ||--o{ B`	One-to-many
`A ||--|| B`	One-to-one
`A }o--o{ B`	Many-to-many
`A |o--|| B`	Optional one-to-one
`A ||--o{ A`	Self-reference
`A ||--o{ B : "has many"`	Quoted label

WORD ALIASES

WORDS	SAME AS
`zero or one`	`|o` / `o|`
`only one / 1`	`||`
`zero or more / 0+`	`}o` / `o{`
`one or more / 1+`	`}|` / `|{`
`many(0) / many(1)`	zero / one or more
`to`	`--` identifying
`optionally to`	`..` non-identifying

⚠ COMMON MISTAKES

- Reading the marker backwards. In `A ||--o{ B` the `o{` sits next to B, so it says how many **B** each A has.
- Leaving out the label. `A ||--o{ B` alone is a syntax error; add `: has` (or `: ""`).
- Using `..` just to make a line look lighter. Dashed means non-identifying: the child can exist without the parent.

✓ TIPS & BEST PRACTICES

- Resolve many-to-many with a join entity (`ORDER_LINE`) — it matches how the tables are really built.
- Write labels as verbs that read left to right: `CUSTOMER places ORDER` .
- Keep the parent entity on the left so every line reads the same way.

QUICK SUMMARY

- `A <m><line><m> B : label` — relationship statement
- `|o || }o }|` — zero/one, one, zero+, one+
- `--` / `..` — identifying / non-identifying
- `: label` — required, reads left to right

State — States & Transitions

STATES AND TRANSITIONS

A `stateDiagram-v2` shows the **states** something can be in and the **transitions** between them. `s1 --> s2 : event` draws a labeled arrow; `[*]` is the start state when it is on the left of an arrow and the end state when it is on the right. A state is created the first time you mention it; give it a longer description with `state "…" as id` or `id : description`. Since v12 state diagrams default to the `redux-color` theme and `neo` look.

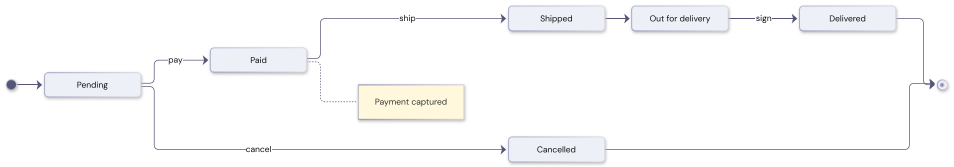
NOTES

- Use `stateDiagram-v2`; `stateDiagram` is the older renderer
- `[*] --> A` = start; `A --> [*]` = end
- Names with spaces need an id: `state "Out for delivery" as ofd`
- Notes can span lines with `note left of X … end note`

ORDER LIFECYCLE

```
stateDiagram-v2
  direction LR
  [*] --> Pending
  Pending --> Paid : pay
  Pending --> Cancelled : cancel
  Paid --> Shipped : ship
  state "Out for delivery" as ofd
  Shipped --> ofd
  ofd --> Delivered : sign
  Delivered --> [*]
  Cancelled --> [*]
  note right of Paid : Payment captured
```

RENDERED OUTPUT



STATES

SYNTAX	MEANING
`Idle`	State (created on first use)
`Idle : Waiting for input`	Add a description
`state "Long name" as s1`	Id with display name
`[*]`	Start / end pseudo-state
`direction LR`	TB (default), LR, BT, RL
`%% text`	Comment

TRANSITIONS

SYNTAX	MEANING
`A --> B`	Transition
`A --> B : event`	Labeled transition
`[*] --> A`	A is the initial state
`A --> [*]`	A is a final state
`A --> A : retry`	Self-transition
`A --> B : evt [guard] / act`	Event, guard, action (text)

NOTES

SYNTAX	MEANING
`note right of A : text`	One-line note
`note left of A`	Start a multi-line note
`…lines…`	Note body
`end note`	Close the note

⚠ COMMON MISTAKES

- Using plain `stateDiagram`. It still parses, but `stateDiagram-v2` is the maintained renderer and supports more syntax.
- Writing a state name with spaces directly (`Out for delivery --> Done`). Declare `state "Out for delivery" as ofd` first.
- Expecting `[*]` to be one shared node. Each `[*]` inside a composite state is its own start or end.

✓ TIPS & BEST PRACTICES

- Label every transition with the event that causes it; unlabeled arrows hide the important part.
- Use `direction LR` for linear lifecycles and the default top-down for branching ones.
- Keep ids short and lowercase, descriptions human: `ofd : Out for delivery`.

QUICK SUMMARY

- `stateDiagram-v2` — declaration
- `A --> B : event` — labeled transition
- `[*]` — start (left of arrow) or end (right of arrow)
- `state "…" as id` — long names
- `note right of A : …` — notes

State — Composite, Choice & Concurrency

NESTED AND PARALLEL STATES

A **composite state** holds its own sub-diagram: `state Active { … }`, with its own `[*]` start and end. **Choice** points (`<<choice>>`) branch on a condition, and **fork / join** bars (`<<fork>>`, `<<join>>`) split into parallel paths and merge them again. Inside a composite state, a line of `--` separates **concurrent regions** that run at the same time. Style states with `classDef` and `class` or the `:::` shortcut.

NOTES

- Composite states can nest several levels deep
- You can transition into or out of the composite as a whole
- Choice / fork / join are declared with `state id <<type>>`
- `classDef` can't style the start / end states

CHECKOUT WITH PARALLEL STEPS

```
stateDiagram-v2
  [*] --> Cart
  Cart --> check
  state check <<choice>>
  check --> Checkout : items > 0
  check --> Cart : empty
  state Checkout {
    [*] --> Address
    --
    [*] --> Payment
  }
  Checkout --> Done
  Done --> [*]
  classDef ok fill:#dff5e1,stroke:#2e7d32
  class Done ok
```

RENDERED OUTPUT



COMPOSITE & REGIONS

SYNTAX	MEANING
`state Active { … }`	Composite state
`state "Long" as L { … }`	Composite with a display name
`[*] --> First`	Start inside the composite
`--`	Separator between concurrent regions
`Active --> Done`	Leave the composite
`direction LR (inside)`	Direction for that composite

CHOICE, FORK & JOIN

SYNTAX	MEANING
`state c <<choice>>`	Decision diamond
`c --> A : x > 0`	Guarded branch
`state f <<fork>>`	Split into parallel paths
`state j <<join>>`	Merge parallel paths
`f --> A / f --> B`	Paths leave the fork
`A --> j / B --> j`	Paths enter the join

STYLING STATES

SYNTAX	MEANING
`classDef bad fill:#fdd`	Define a class
`class A, B bad`	Apply to states
`A:::bad --> B`	Apply inline
`classDef bad,warn …`	Define two at once
`style A fill:#eef`	Style one state

⚠ COMMON MISTAKES

- Forgetting the inner `[*]` in a composite state. Without it, the region has no visible starting point.
- Putting `--` outside a composite state. Concurrency separators only work inside `state X { }`.
- Linking a fork to a join with no states between them — the bars overlap and the diagram becomes unreadable.

✓ TIPS & BEST PRACTICES

- Use a composite state when the same group of states is entered and left as one unit, such as `Checkout`.
- Prefer `<<choice>>` with guard labels over two unlabeled arrows from a normal state.
- Define one `classDef` per meaning (error, done) and reuse it, rather than styling states one by one.

QUICK SUMMARY

- `state X { … }` — composite state with its own `[*]`
- `<<choice>>` — branch on a condition
- `<<fork>>` / `<<join>>` — split and merge
- `--` — concurrent regions inside a composite
- `classDef` + `class` / `:::` — style states

Class — Classes & Members

classDiagram · class Name { } · + - # ~ · $ static / * abstract · ~T~ · <<interface>>

CLASSES AND MEMBERS

A `classDiagram` describes types in object-oriented code. Define a class with `class Name { … }` and list **members** inside, one per line: attributes as `+type name` and methods with parentheses `+name(args) returnType`. A leading symbol sets **visibility**; a trailing `$` marks static and `*` abstract. Generics use tildes (`List~int~`), and annotations like `<<interface>>` describe the kind of class.

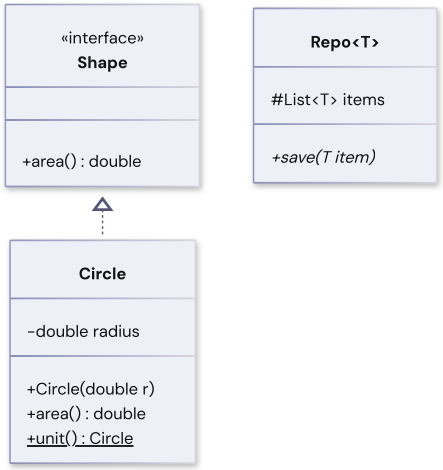
RULES

- Members can also be added one per line: `Account : +deposit(amt)`
- Anything with `()` is a method; everything else is an attribute
- Nested generics work: `List~List~int~~`
- Group classes with `namespace Name { … }`
- v12 default theme: `redux-color`, look `neo`

SHAPES

```
classDiagram
  class Shape {
    <<interface>>
    +area() double
  }
  class Circle {
    -double radius
    +Circle(double r)
    +area() double
    +unit()$ Circle
  }
  class Repo~T~ {
    #List~T~ items
    +save(T item)*
  }
  Shape <|.. Circle
```

RENDERED OUTPUT



MEMBERS

SYNTAX	MEANING
`class Account`	Empty class
`class Account { … }`	Class with members
`+String owner`	Public attribute
`+deposit(amount) bool`	Method with return type
`Account : +balance`	Add member outside the block
`+count$`	Static (underlined)
`+draw()*`	Abstract (italic)
`class A["Display Name"]`	Class label

VISIBILITY & GENERICS

SYNTAX	MEANING
`+`	Public
`-`	Private
`#`	Protected
`~`	Package / internal
`class Box~T~`	Generic class
`List~int~ ids`	Generic type
`Map~K, V~`	Two type parameters

ANNOTATIONS & NAMESPACES

SYNTAX	MEANING
`<<interface>>`	Interface
`<<abstract>>`	Abstract class
`<<enumeration>>`	Enum (values as members)
`<<service>>`	Any stereotype text
`class S <<interface>>`	Inline annotation
`namespace Billing { … }`	Group classes
`namespace B["Billing"]`	Namespace label (v11.15)

⚠ COMMON MISTAKES

- Putting the type after the name for attributes (`name String`). Mermaid draws exactly what you write; pick one style and keep it.
- Using `<T>` for generics. Angle brackets clash with the syntax; write `~T~` .
- Adding a space before `$` or `*` (`area() $`). The marker must touch the closing parenthesis or name.

✓ TIPS & BEST PRACTICES

- Show only the public API on overview diagrams; private fields make boxes long and change often.
- Use `<<interface>>` plus a realization arrow `..|>` to show what implements what (sheet 14).
- Write enums as `<<enumeration>>` with one value per line.

QUICK SUMMARY

- `class Name { … }` — class with members
- `+ - # ~` — public, private, protected, package
- `()$` / `()*` — static / abstract
- `~T~` — generics
- `<<interface>>` — annotations

Class — Relationships & Cardinality

RELATIONSHIPS BETWEEN CLASSES

A relationship line connects two classes: A `<|--` B means B inherits from A. The arrow end shows the **kind** of relationship (inheritance, composition, aggregation, association, dependency, realization) and the line shows solid (`--`) or dashed (`..`). Add a **label** after a colon, **cardinality** in quotes next to each class, and **notes** with `note for`. Arrowheads can point either way.

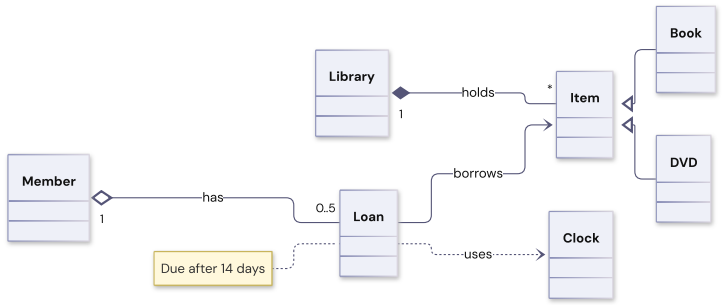
NOTES

- Solid `--` = structural link; dashed `..` = uses / implements
- Cardinality strings go next to the class they describe
- Two-way arrows: put a marker on both ends, `<|--|>`
- `bar ()-- foo` draws a lollipop interface

LIBRARY

```
classDiagram
  direction LR
  Item <|-- Book
  Item <|-- DVD
  Library "1" *-- "*" Item : holds
  Member "1" o-- "0..5" Loan : has
  Loan --> Item : borrows
  Loan ..> Clock : uses
  note for Loan "Due after 14 days"
```

RENDERED OUTPUT



RELATIONSHIP TYPES

SYNTAX	MEANING
`A <|-- B`	Inheritance: B extends A
`A *-- B`	Composition: B dies with A
`A o-- B`	Aggregation: A has B
`A --> B`	Association
`A -- B`	Link (solid)
`A ..> B`	Dependency: A uses B
`A ..|> B`	Realization: A implements B
`A .. B`	Link (dashed)
`A ()-- B`	Lollipop interface A on B

LABELS & CARDINALITY

SYNTAX	MEANING
`A --> B : uses`	Label
`A "1" --> "*" B`	Cardinality both ends
`"0..1"`	Zero or one
`"1..*"`	One or more
`"n"`	Exactly n
`A <|--|> B`	Two-way relationship
`B --|> A`	Same as A `<|--` B

NOTES, LINKS & LAYOUT

SYNTAX	MEANING
`note "text"`	Free-floating note
`note for A "text"`	Note attached to a class
`direction LR`	TB (default), BT, LR, RL
`link A "url" "tip"`	Clickable class
`click A href "url"`	Same, click form
`style A fill:#eef`	Style one class

⚠ COMMON MISTAKES

- Reversing inheritance: `Dog <|-- Animal` says Animal extends Dog. The triangle points at the parent.
- Using composition `*--` for any "has a". Composition means the part can't exist without the whole; otherwise use `o--` or `-->`.
- Placing cardinality after the colon. Quotes go next to the class names: `A "1" --> "*" B : label`.

✓ TIPS & BEST PRACTICES

- Keep the parent above or left of children (`direction` helps) so inheritance trees read naturally.
- Use dashed dependency `..>` for "calls" and solid association `-->` for "stores a reference".
- Put business rules in `note for` instead of long labels on lines.

QUICK SUMMARY

- `<|--` / `..|>` — extends / implements
- `*--` / `o--` — composition / aggregation
- `-->` / `..>` — association / dependency
- `"1" --> "*"` — cardinality
- `note for A` — notes on classes

Gantt — Tasks, Sections & Dates

gantt · dateFormat · axisFormat · section · task : id, start, 3d · after · done / crit · milestone

TASKS ON A CALENDAR

A `gantt` chart draws **tasks** as bars on a date axis, grouped into **sections**. Each task line is `Name : metadata`, where the comma-separated metadata holds optional tags (`done`, `active`, `crit`, `milestone`), an optional **id**, a **start** (a date or `after id`) and an **end** (a date or a duration like `3d`). `dateFormat` tells Mermaid how to read your dates; `axisFormat` sets how they print on the axis.

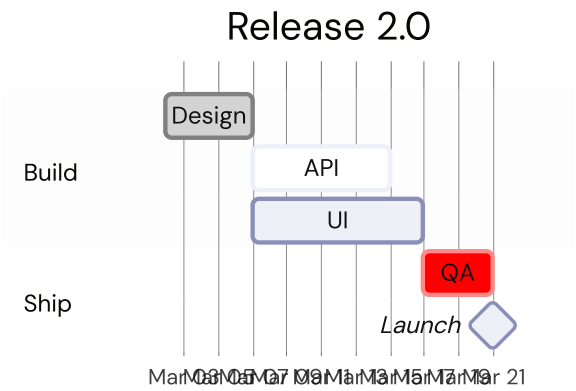
RULES

- Tags come first, then id, then start, then end
- No start given → the task starts when the previous one ends
- `after a b` starts after the **latest** of tasks a and b
- Durations: `ms s m h d w M y` — e.g. `1.5d`
- A milestone is a zero-length task: `:milestone, m1, 2026-06-01, 0d`

RELEASE PLAN

```
gantt
  title Release 2.0
  dateFormat YYYY-MM-DD
  axisFormat %b %d
  section Build
    Design        :done, des, 2026-03-02, 5d
    API           :active, api, after des, 8d
    UI            :ui, after des, 10d
  section Ship
    QA            :crit, qa, after api ui, 4d
    Launch        :milestone, after qa, 0d
```

RENDERED OUTPUT



TASK METADATA

EXAMPLE	MEANING
`:2026-03-02, 5d`	Start date + duration
`:2026-03-02, 2026-03-09`	Start and end dates
`:a1, 2026-03-02, 5d`	With id a1
`:after a1, 3d`	Start when a1 ends
`:after a1 b1, 2d`	After the later of two
`:3d`	Start after previous task
`:2026-03-02, until b1`	Run until b1 starts (v10.9)

TAGS & STATEMENTS

SYNTAX	MEANING
`done`	Finished (grey)
`active`	In progress (highlighted)
`crit`	Critical path (red)
`milestone`	Diamond marker
`crit, done, …`	Tags combine
`title Text`	Chart title
`section Name`	Start a group of rows
`%% comment`	Ignored line

DATE FORMATS

TOKEN	MEANING
`dateFormat YYYY-MM-DD`	Input: Day.js tokens
`dateFormat HH:mm`	Input: times only
`axisFormat %Y-%m-%d`	Axis: d3 time format
`%b %d`	Mar 02
`%a %e`	Mon 2
`%H:%M`	14:30
`%W / %V`	Week number

⚠ COMMON MISTAKES

- Mixing up `dateFormat` (Day.js tokens like `YYYY`) and `axisFormat` (d3 tokens like `%Y`). They use different token systems.
- Putting the id after the start date (`:2026-03-02, des, 5d`). The order is tags, id, start, end.
- Referencing `after x` before task `x` is defined, or with a typo — Mermaid can't place the task and the chart fails.

✓ TIPS & BEST PRACTICES

- Give every task an id even if nothing references it yet; adding dependencies later is then one word.
- Prefer `after id` over hard-coded dates so one slipped task moves everything that depends on it.
- Mark only the true critical path `crit`; when everything is red nothing stands out.

QUICK SUMMARY

- `gantt` + `dateFormat` — declaration and input dates
- `Task :tags, id, start, end` — task line
- `after id` — dependency
- `done` · `active` · `crit` · `milestone` — tags
- `section` — group rows

Gantt Scheduling & Timeline

excludes weekends · tickInterval · todayMarker · vert · timeline · section · period : event

WORKING DAYS, MARKERS & TIMELINES

Gantt charts can skip non-working days with `excludes`, control axis ticks with `tickInterval`, style or hide the **today** line, and add **vertical markers** (`vert`) for deadlines. A `timeline` is the simpler cousin: no dates or durations, just **periods** (any text) with one or more **events**, optionally grouped into sections. Use Gantt for plans you schedule, timeline for history you tell.

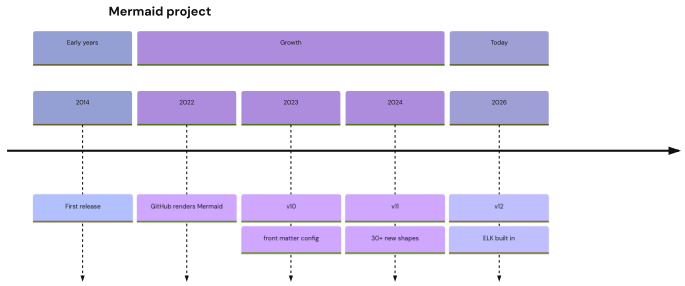
NOTES

- `excludes` takes dates, day names or `weekends` — not `weekdays`
- Excluded days extend a task's bar instead of being counted
- `weekend friday` makes Friday–Saturday the weekend (v11)
- Timeline text is free-form: `2024 Q1`, `Stone Age`, `v1.0`
- `timeline TD` draws top-down (v11.14)

PROJECT HISTORY

```
timeline
  title Mermaid project
  section Early years
    2014 : First release
  section Growth
    2022 : GitHub renders Mermaid
    2023 : v10 : front matter config
    2024 : v11 : 30+ new shapes
  section Today
    2026 : v12 : ELK built in
```

RENDERED OUTPUT



GANTT CALENDAR

SYNTAX	EFFECT
`excludes weekends`	Skip Sat–Sun
`excludes 2026-12-25`	Skip a date
`excludes sunday`	Skip a weekday
`weekend friday`	Weekend = Fri–Sat
`tickInterval 1week`	One tick per week
`weekday monday`	Weeks start on Monday
`todayMarker off`	Hide today line
`todayMarker stroke:#f00`	Style today line

MARKERS & CLICKS

SYNTAX	EFFECT
`Freeze :vert, v1, 2026-04-01, 0d`	Vertical line, no row
`Go live :milestone, m1, …, 0d`	Diamond on its own row
`click t1 href "url"`	Task opens a link
`click t1 call fn()`	Task runs a function
`displayMode: compact`	Config: tasks share rows
`topAxis: true`	Config: axis on top

TIMELINE SYNTAX

SYNTAX	MEANING
`timeline`	Declaration (`timeline TD` top-down)
`title Text`	Title
`section Name`	Group periods
`2020 : Event`	Period with one event
`2020 : A : B`	Several events
`: C`	Continue the previous period
`text more`	Line break

⚠ COMMON MISTAKES

- Writing `excludes weekdays` to show weekends only. Only `weekends`, day names and dates are allowed.
- Using a timeline when tasks have real start and end dates — timelines don't scale by time; use Gantt.
- Indenting timeline events under a period on a new line without the leading `:`. Continuation lines must start with `:`.

✓ TIPS & BEST PRACTICES

- Add `excludes weekends` to any work plan so durations mean working days.
- Use `vert` for fixed dates (code freeze, conference) that aren't tasks.
- Keep timeline periods short (`2024`, `Q3`) and put detail in the events.

QUICK SUMMARY

- `excludes` / `weekend` — working calendar
- `tickInterval` / `axisFormat` — axis control
- `vert` / `todayMarker` — reference lines
- `timeline` + `period : event` — simple history

Mindmap — Structure & Shapes

INDENTATION IS THE STRUCTURE

A `mindmap` grows branches out of one **root** idea. There are no arrows to write: **indentation** alone decides which node is a child of which. Each line is one node; the text can sit in brackets that choose its **shape**, the same idea as flowchart shapes but with a slightly different set. Mindmaps are laid out with the `cose-bilkent` force layout by default, and each top-level branch gets its own color.

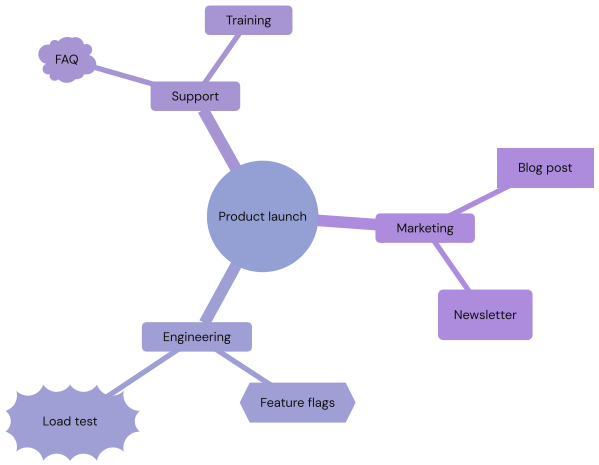
RULES

- Exactly one root: the first node, least indented
- Deeper indentation = child of the line above it
- Indent consistently — spaces, not a mix of tabs and spaces
- Optional id before the shape: `n1[Text]`
- Not in the tiny build

LAUNCH PLAN

```
mindmap
  root((Product launch))
    Marketing
      [Blog post]
      (Newsletter)
    Engineering
      {{Feature flags}}
      ))Load test((
    Support
      )FAQ(
      Training
```

RENDERED OUTPUT



NODE SHAPES

SYNTAX	SHAPE
`Text`	Default (rounded box)
`[Text]`	Square
`(Text)`	Rounded square
`((Text))`	Circle
`))Text((`	Bang
`)Text(`	Cloud
`{{Text}}`	Hexagon
`id[Text]`	Shape with an id

STRUCTURE

PATTERN	RESULT
`mindmap`	Declaration
`root((Idea))`	Root node
`Branch`	Child of root
`Leaf`	Child of Branch
`Branch 2`	Sibling of Branch
`"`**md** text`"`	Markdown string label

CONFIG

MINDMAP: KEY	EFFECT
`colorBy: depth`	Color by level (v12.2)
`padding: 10`	Space inside nodes
`maxNodeWidth: 200`	Wrap width
`layout: tidy-tree`	Tidy tree (plugin)
`look: handDrawn`	Sketch style

⚠ COMMON MISTAKES

- Adding a second root by out-denting a later line. Mermaid throws an error; there can be only one root.
- Mixing tabs and spaces. Two lines that look aligned can parse at different depths.
- Writing `-->` arrows between nodes. Mindmaps have no link syntax; hierarchy comes only from indentation.

✓ TIPS & BEST PRACTICES

- Indent with two spaces per level and turn on visible whitespace in your editor.
- Use shapes for meaning, not decoration: e.g. hexagons for risks, clouds for open questions.
- Keep labels to a few words; long sentences make the force layout spread out.

QUICK SUMMARY

- `mindmap` — declaration
- indentation — the only way to make children
- `root((…))` — one root node
- `[ ] ( ) (( )) )) (( ) ( {{ }}` — shapes

Mindmap — Icons, Classes & Layout

DECORATING NODES

Mindmap nodes can carry an **icon** and **CSS classes**, each written on its own line directly under the node: `::icon(fa fa-book)` and `:::urgent`. Icons use icon-font classes (Font Awesome, Material Design Icons) that the page must load. **Markdown strings** give bold, italic and automatic wrapping. Config keys control colors by depth and the layout engine; `tidy-tree` gives a neat, balanced tree when its plugin is registered.

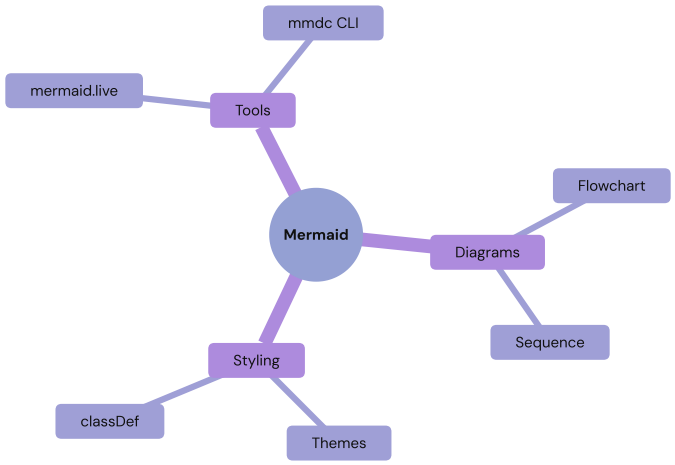
NOTES

- `::icon()` and `:::` lines go under the node they decorate
- Icon fonts must be loaded by the page, not the diagram
- `:::` adds page CSS classes — define them in the site's CSS
- Mindmap icon syntax is still marked experimental
- Indentation rules from sheet 17 still apply

STUDY GUIDE

```
---
config:
  mindmap:
    colorBy: depth
---
mindmap
  root(("`**Mermaid**`"))
    Diagrams
      Flowchart
      Sequence
    Styling
      Themes
      classDef
    Tools
      mermaid.live
      mmdc CLI
```

RENDERED OUTPUT



ICONS & CLASSES

SYNTAX	EFFECT
`::icon(fa fa-book)`	Font Awesome icon
`::icon(mdi mdi-skull)`	Material Design icon
`:::urgent`	Add CSS class `urgent`
`:::urgent large`	Several classes
`"`**bold** _it_`"`	Markdown string
`Line one two`	Manual line break

LAYOUT & LOOK

CONFIG	EFFECT
`(default)`	cose-bilkent force layout
`layout: tidy-tree`	Balanced tree (plugin)
`mindmap.colorBy: depth`	Same color per level
`look: handDrawn`	Sketchy outlines
`theme: forest`	Any theme applies

GOOD USES

USE	PATTERN
`Brainstorm`	Root = question, branches = ideas
`Study notes`	Root = topic, leaves = facts
`Project scope`	Branches = workstreams
`Meeting agenda`	Branches = agenda items
`Feature map`	Branches = user goals

⚠ COMMON MISTAKES

- Putting `::icon()` on the same line as the node text. It must be its own line, indented under the node.
- Expecting icons to appear on GitHub: the platform doesn't load Font Awesome, so the icon slot stays empty.
- Using `layout: tidy-tree` without registering `@mermaid-js/layout-tidy-tree`. Mermaid falls back to another layout and logs a warning.

✓ TIPS & BEST PRACTICES

- Use `colorBy: depth` when levels matter more than branches (outlines, study guides).
- Use markdown strings for long node text so it wraps instead of making very wide nodes.
- Export big mindmaps with `mmdc -s 2` for crisp PNGs in slides.

QUICK SUMMARY

- `::icon(…)` — icon under a node
- `:::class` — CSS classes under a node
- `" … "` — markdown strings
- `colorBy` / `layout` — color and layout config

Sheet 19

Git Graph — Commits & Branches

gitGraph · commit id: / tag: · type: · branch · checkout / switch · order:

DRAWING GIT HISTORY

A `gitGraph` replays Git commands to draw history. It starts on `main`; `commit` adds a commit to the current branch, `branch name` creates a branch **and switches to it**, and `checkout name` (or `switch name`) moves back to an existing branch. Commits can get a custom **id**, a **tag**, and a **type** that changes the dot (normal, reverse, highlight). Commands run top to bottom, exactly as you would type them.

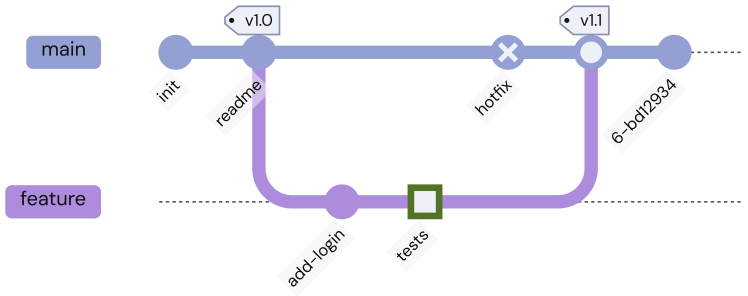
RULES

- The first branch is `main` (rename with `mainBranchName`)
- `branch` both creates and checks out the new branch
- `checkout` and `switch` are interchangeable
- Without `id:`, commits get random ids
- Quote ids and tags: `id: "fix-1"`

FEATURE BRANCH

```
gitGraph
  commit id: "init"
  commit id: "readme" tag: "v1.0"
  branch feature
  commit id: "add-login"
  commit id: "tests" type: HIGHLIGHT
  checkout main
  commit id: "hotfix" type: REVERSE
  merge feature tag: "v1.1"
  commit
```

RENDERED OUTPUT



COMMITS

SYNTAX	EFFECT
`commit`	New commit on current branch
`commit id: "abc"`	Custom id (shown as label)
`commit tag: "v1.0"`	Tag label
`commit type: NORMAL`	Filled dot (default)
`commit type: REVERSE`	Crossed dot (revert)
`commit type: HIGHLIGHT`	Square, highlighted
`commit id: "x" tag: "t"`	Attributes combine

BRANCHES

SYNTAX	EFFECT
`branch develop`	Create + switch to `develop`
`branch hotfix order: 3`	Fix display position
`checkout main`	Switch to existing branch
`switch main`	Same as `checkout`
`branch "feat/x"`	Quote names with `/`

ORIENTATION

DECLARATION	DIRECTION
`gitGraph`	Left → right (default)
`gitGraph LR:`	Left → right
`gitGraph TB:`	Top → bottom
`gitGraph BT:`	Bottom → top (v11)

⚠ COMMON MISTAKES

- Running `checkout` on a branch that hasn't been created. Mermaid stops with an error; create it with `branch` first.
- Writing `branch x` and then `checkout x` again — `branch` already switched. The extra checkout is harmless but hides intent.
- Forgetting the colon in orientation: `gitGraph TB` instead of `gitGraph TB:`.

✓ TIPS & BEST PRACTICES

- Give meaningful commits an `id:` — merges and cherry-picks (sheet 20) need ids to refer to.
- Use `type: HIGHLIGHT` to point at the commit a discussion is about.
- Use `order:` to keep long-lived branches (main, develop) next to each other.

QUICK SUMMARY

- `gitGraph` — starts on `main`
- `commit id: tag: type:` — commits
- `branch` — create and switch
- `checkout` / `switch` — move between branches
- `LR: TB: BT:` — orientation

Git Graph — Merges, Cherry-Picks & Options

MERGING AND CONFIG

`merge name` joins another branch into the current one and creates a **merge commit**, which can take its own `id`, `tag` and `type`. `cherry-pick id: "…"` copies one commit from another branch onto the current one; for a merge commit you must also give `parent:`. Front matter `config: gitGraph:` controls the branch name, labels, ordering and whether parallel commits line up.

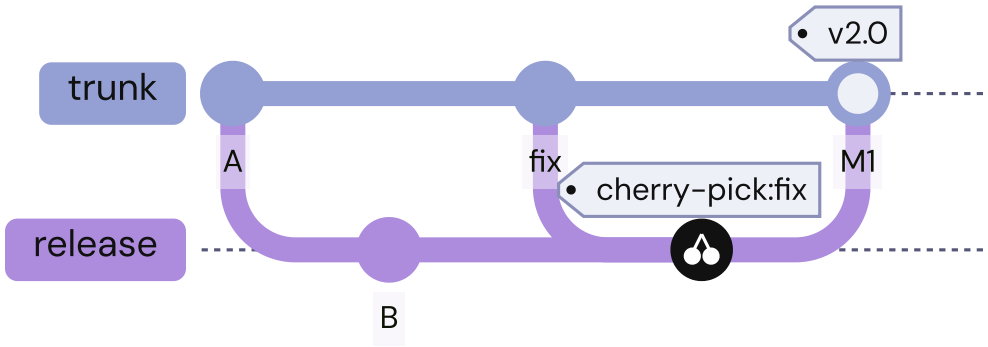
RULES

- You can't merge a branch into itself
- Cherry-pick source must be on a **different** branch
- The current branch needs a commit before a cherry-pick
- Branch colors come from `git0` … `git7` theme variables

RELEASE BRANCH

```
---
config:
  gitGraph:
    mainBranchName: trunk
    rotateCommitLabel: false
---
gitGraph
  commit id: "A"
  branch release
  commit id: "B"
  checkout trunk
  commit id: "fix"
  checkout release
  cherry-pick id: "fix"
  checkout trunk
  merge release id: "M1" tag: "v2.0"
```

RENDERED OUTPUT



MERGE & CHERRY-PICK

SYNTAX	EFFECT
`merge develop`	Merge develop into current
`merge develop id: "M1"`	Merge commit id
`merge develop tag: "v2"`	Tag the merge
`merge develop type: REVERSE`	Merge commit type
`cherry-pick id: "abc"`	Copy commit abc here
`cherry-pick id: "M1" parent: "B"`	Pick a merge commit
`cherry-pick id: "abc" tag: "t"`	Custom tag on the copy

GITGRAPH: CONFIG

KEY	EFFECT
`mainBranchName: trunk`	Rename `main`
`mainBranchOrder: 2`	Position of main
`showBranches: false`	Hide branch lines / names
`showCommitLabel: false`	Hide commit labels
`rotateCommitLabel: false`	Horizontal labels
`parallelCommits: true`	Align commits by distance

THEME VARIABLES

VARIABLE	COLORS
`git0 … git7`	Branch lines
`gitBranchLabel0 … 7`	Branch label text
`gitInv0 … 7`	Highlight commits
`commitLabelColor`	Commit label text
`tagLabelBackground`	Tag background
`commitLabelFontSize`	Label size

⚠ COMMON MISTAKES

- Cherry-picking a commit that's already on the current branch. Mermaid throws an error; pick from another branch.
- Cherry-picking a merge commit without `parent:`. A merge has two parents, so Mermaid needs to know which.
- Expecting `merge` to fast-forward. Mermaid always draws a merge commit, even when Git wouldn't.

✓ TIPS & BEST PRACTICES

- Use `rotateCommitLabel: false` with short ids; rotated labels are easier only for long ones.
- Tag merges, not ordinary commits, for release diagrams: `merge release tag: "v2.0"`.
- Hide labels (`showCommitLabel: false`) when the shape of history is the point, not the ids.

QUICK SUMMARY

- `merge name` — merge commit on current branch
- `cherry-pick id:` (`parent:`) — copy a commit
- `mainBranchName` — rename main
- `showBranches` / `showCommitLabel` — hide parts
- `git0…git7` — branch colors

Themes, Looks & Configuration

config: · theme · look: classic / handDrawn / neo · layout: dagre / elk · %%{init}%% · priority

CHANGING THE WHOLE DIAGRAM

Three settings change how an entire diagram looks: **theme** (colors), **look** (line style: `classic`, `handDrawn`, `neo`) and **layout** (the engine that places nodes). Set them for one diagram in **front matter** under `config:`, or for a whole site in `mermaid.initialize()`. Since v12, ten diagram types (flowchart, sequence, class, state, ER, requirement, use case, venn, swimlane, agentflow) default to the `redux-color` theme and `neo` look; the rest use `default` and `classic`.

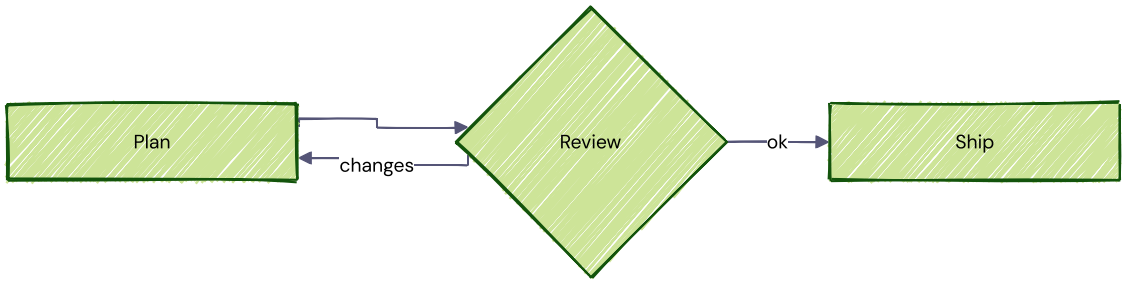
NOTES

- Front matter `config:` is the supported per-diagram method
- Scope a setting to one type: `config: flowchart: look: handDrawn`
- ELK is bundled in v12; `layout: dagre` gives the older layout
- Get the pre-v12 look back: `theme: default` + `look: classic`
- Platforms (GitHub, Obsidian) may ignore or limit config

FRONT MATTER CONFIG

```
---
config:
  theme: forest
  look: handDrawn
  layout: dagre
  flowchart:
    curve: stepBefore
---
flowchart LR
  A[Plan] --> B{Review}
  B -->|ok| C[Ship]
  B -->|changes| A
```

RENDERED OUTPUT



THEMES

THEME:	LOOK
`default`	Classic Mermaid colors
`neutral`	Greys — good for print
`dark`	For dark pages
`forest`	Greens
`base`	Only theme you can customize
`neo / neo-dark`	Flat, soft; pair with `look: neo`
`redux / redux-dark`	Monochrome v12 style
`redux-color`	v12 default for 10 types
`redux-dark-color`	Dark version

LOOKS & LAYOUTS

SETTING	EFFECT
`look: classic`	Traditional lines
`look: handDrawn`	Sketchy, whiteboard style
`look: neo`	Modern, rounded
`layout: dagre`	Layered layout (pre-v12)
`layout: elk`	ELK layered (bundled v12)
`layout: elk.stress`	ELK variant (also `.force`, `.mrtree`)
`layout: cose-bilkent`	Force-directed (mindmap default)
`layout: tidy-tree`	Tree layout (plugin)

WHO WINS (HIGHEST FIRST)

SOURCE	EXAMPLE
`1. front matter`	`config: theme: dark`
`1. %%{init}%%`	Deprecated directive, same level
`2. initialize()`	`mermaid.initialize({ theme })`
`3. type default`	v12 per-diagram defaults
`4. global default`	`default` · `classic` · `dagre`
`scoped key`	`flowchart: { look }` beats global

⚠ COMMON MISTAKES

- Customizing `themeVariables` on a theme other than `base`. Only `base` can be modified; other themes ignore most variables.
- Indenting front matter keys inconsistently. It's YAML: `theme:` must sit two spaces under `config:`.
- Relying on `%%{init}%%` in new docs. It's deprecated since v10.5; use front matter.

✓ TIPS & BEST PRACTICES

- Use `theme: neutral` for documents that will be printed in black and white.
- Set site-wide choices once in `initialize()` and keep front matter for exceptions.
- Pin `theme`, `look` and `layout` explicitly in docs you publish so a Mermaid upgrade doesn't change them.

QUICK SUMMARY

- `config:` — per-diagram settings in front matter
- `theme` — colors (`base` is customizable)
- `look` — `classic`, `handDrawn`, `neo`
- `layout` — `dagre`, `elk`, `cose-bilkent`, …
- `initialize()` — site-wide defaults

Styling Elements & Theme Variables

STYLING PARTS OF A DIAGRAM

To style **individual elements**, define a reusable class with `classDef name props` and attach it with `class id name` or `id:::name`; use `style id props` for a one-off and `linkStyle n props` for links. Props are CSS-like (`fill`, `stroke`, `color`). To restyle **everything** at once, use the `base` theme and override **theme variables** in front matter; Mermaid derives the remaining colors from the ones you set.

NOTES

- Theme variables accept **hex colors only** — `#ff0000`, not `red`
- `classDef default …` styles every node without a class
- Separate props with commas; escape commas inside values as `\,`
- Multiple classes: `classDef a,b fill:#eee`

CLASSES, STYLES AND VARIABLES

```
---
config:
  theme: base
  themeVariables:
    primaryColor: "#e8f0fe"
    primaryBorderColor: "#3367d6"
    lineColor: "#5f6368"
---
flowchart LR
  A[Draft]:::muted --> B[Review] --> C[Live]
  classDef muted fill:#f1f3f4,color:#80868b
  classDef go fill:#d4edda,stroke:#28a745
  class C go
  style B stroke-dasharray: 5 5
  linkStyle 1 stroke:#28a745,stroke-width:2px
```

RENDERED OUTPUT



ELEMENT STYLING

SYNTAX	EFFECT
`classDef hot fill:#fdd`	Define a class
`class A,B hot`	Apply to several ids
`A:::hot`	Apply inline
`classDef default …`	Style all unclassed nodes
`style A fill:#eef`	Style one element
`linkStyle 0,2 stroke:#999`	Style links by index
`linkStyle default …`	Style all links

STYLE PROPERTIES

PROPERTY	EXAMPLE
`fill`	`fill:#fff3cd`
`stroke`	`stroke:#856404`
`stroke-width`	`stroke-width:2px`
`stroke-dasharray`	`stroke-dasharray: 5 5`
`color`	`color:#000` (text)
`font-weight`	`font-weight:bold`
`font-size`	`font-size:14px`

KEY THEME VARIABLES

VARIABLE	CONTROLS
`primaryColor`	Node fill; others derive from it
`primaryTextColor`	Text on primary nodes
`primaryBorderColor`	Node borders
`secondaryColor`	Secondary fills
`lineColor`	Links and arrows
`background`	Diagram background
`fontFamily` / `fontSize`	Text
`darkMode: true`	Derive colors for dark pages

⚠ COMMON MISTAKES

- Writing color names in `themeVariables` (`primaryColor: blue`). The theme engine only understands hex.
- Using `classDef` in diagrams that don't support it. It works in flowchart, state, class, ER, requirement, block, use case and a few others — not sequence or gantt.
- Counting `linkStyle` from 1. Indexes start at 0 in the order links are written.

✓ TIPS & BEST PRACTICES

- Name classes by meaning (`error`, `done`), not by color, so a palette change is one line.
- Set just `primaryColor` first; Mermaid derives borders and secondary colors to match.
- Prefer edge ids (`e1@{ … }`, sheet 06) over `linkStyle` numbers in diagrams that change often.

QUICK SUMMARY

- `classDef` + `class` / `:::` — reusable styles
- `style id …` — one-off style
- `linkStyle n …` — style links
- `theme: base` + `themeVariables` — global palette
- `#hex` — the only color format theme variables accept

Swimlanes BETA

swimlane-beta · subgraph lanes · flowchart shapes · --> edges · cross-lane handoffs · LR / TB

A FLOWCHART SPLIT BY OWNER

A `swimlane-beta` diagram is a flowchart whose top-level **subgraphs become lanes**, one per person, team or system. Nodes and edges use the flowchart syntax from sheets 05–06, so the new part is only where you declare each node: inside the lane that owns the step. Links that cross a lane boundary are **handoffs**, the places where work changes hands. Swimlanes use their own `swimlane` layout and the v12 `redux-color` / `neo` defaults.

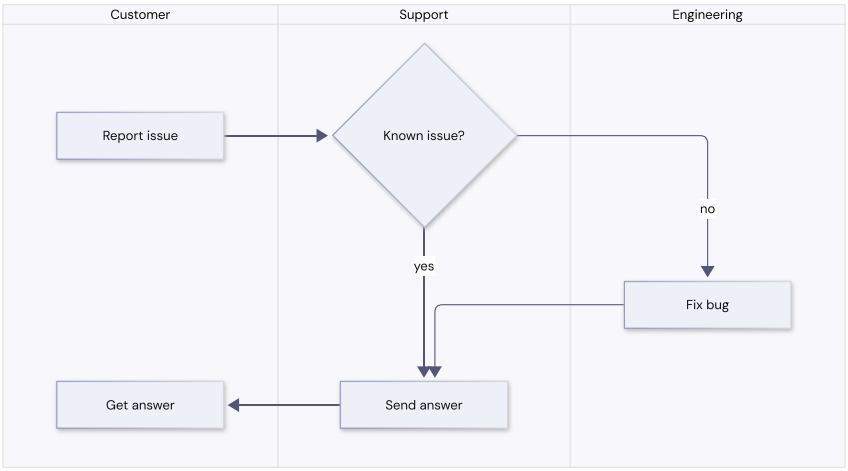
NOTES

- Default direction is `TB` ; `LR` makes horizontal lanes
- Lane id + label: `subgraph ops [Operations]`
- Declare a node inside the lane that owns it
- Supports `accTitle` / `accDescr`
- New in v12 — needs the `-beta` suffix

SUPPORT TICKET

```
swimlane-beta
  subgraph cust [Customer]
    ask[Report issue]
    got[Get answer]
  end
  subgraph sup [Support]
    triage{Known issue?}
    reply[Send answer]
  end
  subgraph eng [Engineering]
    fix[Fix bug]
  end
  ask --> triage
  triage -->|yes| reply
  triage -->|no| fix
  fix --> reply --> got
```

RENDERED OUTPUT



SYNTAX

SYNTAX	MEANING
`swimlane-beta`	Declaration (TB default)
`swimlane-beta LR`	Horizontal lanes
`subgraph Sales … end`	Lane
`subgraph s [Sales team]`	Lane id + label
`id[Text] id{Text}`	Flowchart node shapes
`A -->|label| B`	Flowchart edges

GOOD PRACTICE

DO	WHY
`One kind of owner per lane`	Teams **or** systems, not both
`Label handoff edges`	Shows what is passed on
`Decision in deciding lane`	Shows who decides
`Stable short ids`	Easy to restyle and link
`≤ 5 lanes`	Wider becomes unreadable

SWIMLANE VS OTHERS

NEED	USE
`Steps + owners`	`swimlane-beta`
`Steps only`	`flowchart` (sheet 05)
`Messages over time`	`sequenceDiagram` (07)
`Experience scores`	`journey` (24)

⚠ COMMON MISTAKES

- Using `flowchart` with subgraphs and expecting lanes. Only `swimlane-beta` lays top-level subgraphs out as aligned lanes.
- Declaring a node only in an edge line. It then lands outside every lane; declare it inside its lane first.
- Nesting lanes inside lanes. Only top-level subgraphs become lanes.

✓ TIPS & BEST PRACTICES

- Write the lanes first with their nodes, then all the edges at the bottom — the source mirrors the picture.
- Use `LR` for long processes so lanes run across the page.
- Label every edge that crosses lanes with what is handed over (`ticket` , `invoice`).

QUICK SUMMARY

- `swimlane-beta` — flowchart with lanes
- `subgraph … end` — one lane per owner
- flowchart shapes and edges — same as sheets 05–06
- cross-lane edges — handoffs worth labeling

User Journey

HOW A TASK FEELS

A `journey` diagram maps the steps a user takes to finish a goal and how each step **feels**. Steps are grouped into **sections**; each task line is `Task name: score: actors`, where the score runs from **1** (bad) to **5** (great) and the actors are a comma-separated list. Mermaid draws faces and colors from the scores, so low points stand out at a glance.

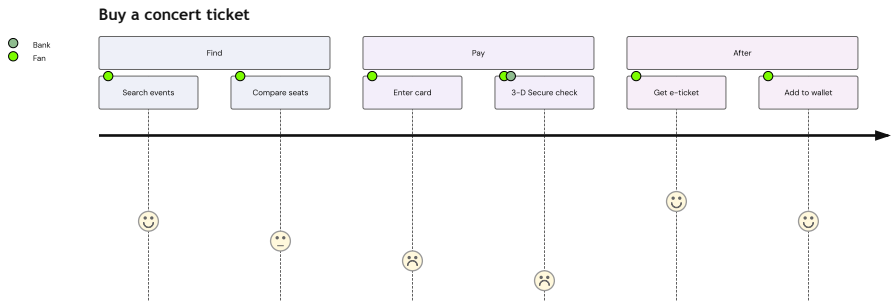
NOTES

- Score must be a whole number from 1 to 5
- Several actors per task: `Pay: 2: Me, Bank`
- Each actor gets a color and a legend entry
- Sections run left to right in order

BUYING A TICKET

```
journey
  title Buy a concert ticket
  section Find
    Search events: 4: Fan
    Compare seats: 3: Fan
  section Pay
    Enter card: 2: Fan
    3-D Secure check: 1: Fan, Bank
  section After
    Get e-ticket: 5: Fan
    Add to wallet: 4: Fan
```

RENDERED OUTPUT



SYNTAX

SYNTAX	MEANING
`journey`	Declaration
`title Text`	Diagram title
`section Name`	Group of steps
`Task: 5: Me`	Task, score, actor
`Task: 3: Me, Cat`	Several actors

SCORES

SCORE	MEANING
5	Delighted
4	Happy
3	Neutral
2	Frustrated
1	Angry / blocked

STYLING

VARIABLE	COLORS
`fillType0 … 7`	Section colors
`journey.actorColours`	Actor colors (config)
`faceColor`	Face fill
`textColor`	Labels

⚠ COMMON MISTAKES

- Using scores outside 1–5 or decimals (`3.5`). The parser rejects them.
- Forgetting the second colon: `Search: Fan` has no score, so the line fails.
- Packing in every click. A journey is about the shape of the experience; ten steps per section is too many.

✓ TIPS & BEST PRACTICES

- Base scores on research (interviews, support tickets), not guesses, and note the source in the doc.
- Add the system as an actor on steps where it's involved (`Fan, Bank`) to show who causes the pain.
- Pair a journey with a flowchart: the journey shows feelings, the flowchart shows the logic.

QUICK SUMMARY

- `journey` + `title` — declaration
- `section` — phases of the journey
- `Task: score: actors` — one step
- `1-5` — score drives faces and colors

Use Case Diagram BETA

ACTORS AND GOALS

A `usecase-beta` diagram shows **actors** (people or systems) and the **use cases** they take part in, usually inside a **system boundary**. Actors must be declared with `actor`; use cases are written with parentheses for the classic ellipse or brackets for a rectangle. UML relationships have explicit operators: `..> : include`, `..> : extend` and `--|>` for generalization. It is one of the v12 types that defaults to `redux-color`, `neo` and ELK.

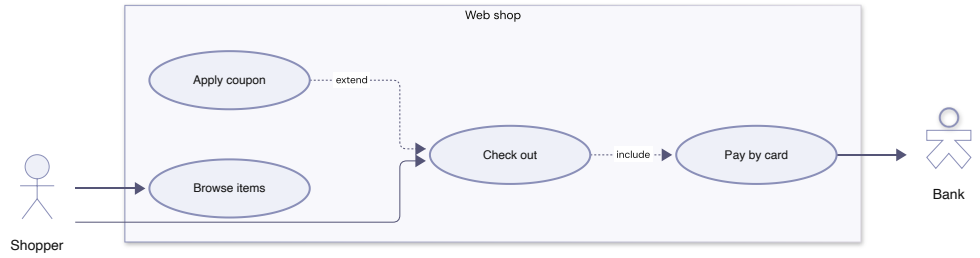
NOTES

- Undeclared endpoints become use cases — actors need `actor`
- `Login("Sign in")` = id `Login`, label `Sign in`
- Boundaries are one level deep and hold only declarations
- Actor types: `@{ type: hollow }`, `awesome`, or `icon`
- `;` is not a separator; one statement per line

ONLINE SHOP

```
usecase-beta
  direction LR
  actor Shopper
  actor Bank@{ type: hollow }
  systemBoundary shop["Web shop"]
    Browse("Browse items")
    Checkout("Check out")
    Pay("Pay by card")
    Coupon("Apply coupon")
  end
  Shopper --> Browse
  Shopper --> Checkout
  Checkout ..> : include Pay
  Coupon ..> : extend Checkout
  Pay --> Bank
```

RENDERED OUTPUT



ELEMENTS

SYNTAX	MEANING
`actor User`	Stick-figure actor
`actor A("Admin")`	Actor id + label
`actor B@{ type: hollow }`	Hollow actor
`actor C@{ icon: "fa:user" }`	Icon actor (icon pack)
`Login("Sign in")`	Ellipse use case
`Report[Run report]`	Rectangle use case
`@{ business: true }`	Business slash
`<<Core>>`	Stereotype label

RELATIONSHIPS

SYNTAX	MEANING
`A --> B`	Association
`A -- B`	Association, no arrow
`A --o B` / `A --x B`	Circle / cross end
`A ..> : include B`	A includes B
`A ..> : extend B`	A extends B
`A --|> B`	Generalization
`A -- "label" --> B`	Labeled association
`A e1@--> B`	Edge id for styling

STRUCTURE

SYNTAX	MEANING
`systemBoundary id[Title]`	Open boundary
`end`	Close boundary
`@{ type: package }`	Boundary with title tab
`direction LR`	Layout direction
`note for Login "…"`	Note on an element
`classDef / class / :::`	Styling

⚠ COMMON MISTAKES

- Expecting `User --> Login` to create an actor. Endpoints without a declaration become use cases; add `actor User`.
- Writing include as a label: `A -- "include" --> B` is an ordinary association. Use `A ..> : include B`.
- Putting relationships inside `systemBoundary`. Boundaries only hold actor and use case declarations.

✓ TIPS & BEST PRACTICES

- Name use cases as verb phrases from the actor's view: `Check out`, `Track order`.
- Put external systems (bank, email) outside the boundary as hollow actors.
- Use `include` for steps that always happen, `extend` for optional ones.

QUICK SUMMARY

- `usecase-beta` — declaration
- `actor` — required for actors
- `( )` / `[ ]` — ellipse / rectangle use cases
- `systemBoundary … end` — the system
- `..> : include` · `..> : extend` · `--|>` — UML relations

Requirement Diagram

TRACING REQUIREMENTS

A `requirementDiagram` (based on SysML v1.6) shows **requirements**, the **elements** that implement or test them, and the **relationships** between them. A requirement block has a type keyword, a name, and fields: `id`, `text`, `risk` and `verifymethod`. An element block has a `type` and a `docref`. Relationship lines read `source - type -> destination`. v12 defaults: `redux-color`, `neo`, ELK.

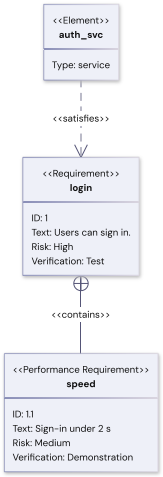
NOTES

- Quote text that contains `-` or keywords: `text: "Sign-in…"`
- Quoted text supports markdown: `text: "**must** be fast"`
- Reverse form: `dest <- type - source`
- `direction LR` changes the layout

LOGIN REQUIREMENTS

```
requirementDiagram
  requirement login {
    id: 1
    text: Users can sign in.
    risk: high
    verifymethod: test
  }
  performanceRequirement speed {
    id: 1.1
    text: "Sign-in under 2 s"
    risk: medium
    verifymethod: demonstration
  }
  element auth_svc {
    type: service
  }
  login - contains -> speed
  auth_svc - satisfies -> login
```

RENDERED OUTPUT



BLOCK TYPES

KEYWORD	MEANING
`requirement`	Generic requirement
`functionalRequirement`	Behavior
`interfaceRequirement`	Interface
`performanceRequirement`	Performance
`physicalRequirement`	Physical
`designConstraint`	Constraint
`element`	Thing that meets / tests it

FIELDS

FIELD	VALUES
`id:`	Any text, e.g. `1.2`
`text:`	Requirement text
`risk:`	`low` · `medium` · `high`
`verifymethod:`	`analysis` · `inspection` · `test` · `demonstration`
`type:`	Element type (free text)
`docref:`	Element doc reference

RELATIONSHIPS

TYPE	MEANING
`contains`	Parent holds child
`copies`	Copy of another
`derives`	Derived from
`satisfies`	Element meets it
`verifies`	Element tests it
`refines`	Adds detail
`traces`	General trace link

⚠ COMMON MISTAKES

- Using unquoted text that contains a keyword (`text: element must load`). The parser sees `element` ; quote the text.
- Writing arrows like flowcharts (`a --> b`). Requirement relationships are `a - satisfies -> b` .
- Inventing risk or verification values. Only the listed values are accepted.

✓ TIPS & BEST PRACTICES

- Use the same ids as your requirements tracker so the diagram links back to it.
- Add `element` blocks for tests (`type: test suite`) and connect them with `verifies` to show coverage.
- Split big specs into one diagram per feature area.

QUICK SUMMARY

- `requirementDiagram` — declaration
- `requirement name { id text risk verifymethod }` — a requirement
- `element name { type docref }` — implementing / testing thing
- `a - satisfies -> b` — relationship

ZenUML

SEQUENCE DIAGRAMS WRITTEN AS CODE

`zenuml` is an alternative sequence-diagram syntax that reads like code. A **sync** call is `A.method(args)`, and braces `{ }` nest the calls it makes. **Async** messages use `A->B: text`, `new A()` creates a participant, and `return` (or `x = A.call()`) draws the reply. Control flow uses familiar keywords: `if/else`, `while`, `opt`, `par`, `try/catch/finally`. ZenUML is an **external plugin**: the page must register `@mermaid-js/mermaid-zenuml`.

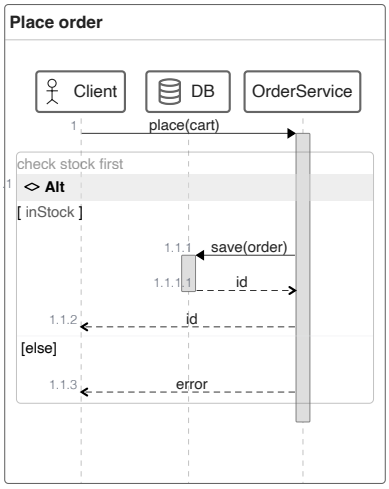
NOTES

- Not in core Mermaid — `registerExternalDiagrams([zenuml])`
- `//` comments above a message are rendered (markdown OK)
- Annotators set the symbol: `@Actor`, `@Database`, …
- `A as Alice` gives an alias
- GitHub and most platforms don't render it

ORDER SERVICE

```
zenuml
  title Place order
  @Actor Client
  @Database DB
  Client->OrderService.place(cart) {
    // check stock first
    if(inStock) {
      id = DB.save(order)
      return id
    } else {
      return error
    }
  }
```

RENDERED OUTPUT



MESSAGES

SYNTAX	MEANING
`A.method()`	Sync call
`A.method(x, y) { … }`	Call with nested calls
`A->B: text`	Async message
`new A()`	Create participant
`x = A.method()`	Reply assigned to x
`return x`	Reply inside a block
`@return A->B: x`	Reply up one level

CONTROL FLOW

SYNTAX	FRAGMENT
`if(c) { } else { }`	alt
`while(c) { }`	loop (`for`, `forEach`, `loop`)
`opt { }`	opt
`par { }`	par
`try { } catch { } finally { }`	break / error path
`// comment`	Rendered comment

PARTICIPANTS

SYNTAX	MEANING
`Alice`	Declare (sets order)
`A as Alice`	Alias
`@Actor A`	Person symbol
`@Database A`	Database symbol
`@Boundary / @Control`	UML stereotypes
`title Text`	Diagram title

⚠ COMMON MISTAKES

- Using ZenUML where only core Mermaid runs (GitHub, mermaid.live without the plugin). It fails with "unknown diagram type".
- Mixing Mermaid sequence syntax (`->>` , `activate`) into ZenUML. The two syntaxes are separate.
- Forgetting braces after a call that makes further calls — nested calls then appear as top-level messages.

✓ TIPS & BEST PRACTICES

- Choose ZenUML when the diagram mirrors real code; choose `sequenceDiagram` (sheet 07) for portability.
- Name calls after real methods (`OrderService.place`) so the diagram stays in sync with the code.
- Use `//` comments for the "why"; they render above the message.

QUICK SUMMARY

- `zenuml` — plugin diagram type
- `A.method() { }` — sync call with nesting
- `A->B: text` — async
- `return` / `x = …` — replies
- `if` · `while` · `try` — control flow

Event Modeling

INFORMATION FLOW OVER TIME

An `eventmodeling` diagram follows the **Event Modeling** method: a timeline of **time frames**, each holding one entity — a UI screen, a processor, a command, an event or a read model. Each line is `tf <number> <type> <Name>`; Mermaid infers the arrow from each frame to the next and puts entities in swimlanes by type. Use `rf` (reset frame) to break the inferred chain and `->>` to link frames explicitly.

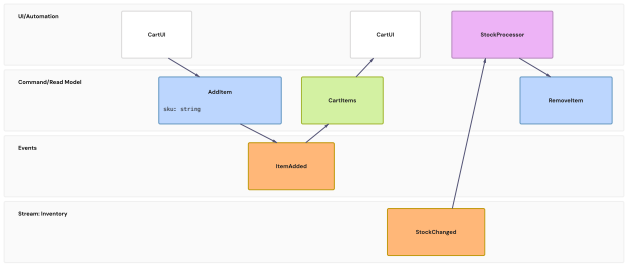
NOTES

- Frame numbers must be unique; their order doesn't matter
- `tf` / `timeframe` (compact / relaxed) are interchangeable
- `Namespace.Name` puts an entity in its own swimlane
- Inline data: `{ description: string }` after the name
- See eventmodeling.org for the method

ADD TO CART

```
eventmodeling
  tf 01 ui CartUI
  tf 02 cmd AddItem { sku: string }
  tf 03 evt ItemAdded
  tf 04 rmo CartItems
  tf 05 ui CartUI
  rf 06 evt Inventory.StockChanged
  tf 07 pcr StockProcessor
  tf 08 cmd RemoveItem
```

RENDERED OUTPUT



ENTITY TYPES

TOKEN	ENTITY · SWIMLANE
`ui`	UI screen · UI/Automation
`pcr` / `processor`	Automation · UI/Automation
`cmd` / `command`	Command · Command/Read Model
`rmo` / `readmodel`	Read model · Command/Read Model
`evt` / `event`	Event · Events

FRAMES & LINKS

SYNTAX	MEANING
`tf 01 ui CartUI`	Time frame
`timeframe 01 ui CartUI`	Same, relaxed form
`rf 04 evt X`	Reset frame: no inferred link
`tf 05 rmo V ->> 02 ->> 03`	Explicit links to frames
`Shop.ItemAdded`	Namespace → own swimlane

DATA

SYNTAX	MEANING
`{ price: number }`	Inline data
`[[AddItem01]]`	Reference a data block
`data AddItem01 { … }`	Data block
`` `json` … `md` ``	Typed data block content

⚠ COMMON MISTAKES

- Reusing a frame number. Every `tf` and `rf` needs its own number, or the diagram errors.
- Expecting an arrow into a reset frame. `rf` deliberately stops the inferred link; add `->>` if one is needed.
- Mixing up commands and events: commands are requests (`AddItem`), events are facts in past tense (`ItemAdded`).

✓ TIPS & BEST PRACTICES

- Number frames in steps of 10 (`10` , `20`) so you can insert frames later without renumbering.
- Name events in past tense and commands as imperatives — the diagram then reads as a story.
- Start with the happy path using `tf` only, then add `rf` branches and data.

QUICK SUMMARY

- `eventmodeling` — declaration
- `tf <n> <type> <Name>` — time frame
- `ui` · `cmd` · `evt` · `rmo` · `pcr` — entity types
- `rf` / `->>` — break or add links

Pie Chart

SHARE OF A WHOLE

A `pie` chart shows how a total splits into parts. After the declaration (optionally `showData` to print values in the legend) and a `title`, each line is a quoted **label**, a colon and a positive **number**. Slices are drawn clockwise in the order you write them. Since v11.16, config can turn it into a donut, move the legend and highlight a slice.

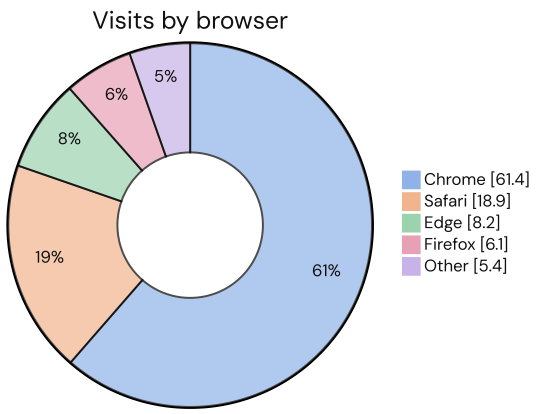
NOTES

- Values must be positive; up to two decimals
- Labels must be in double quotes
- Mermaid works out the percentages
- `title` can sit on the declaration line: `pie title Pets`

BROWSER SHARE

```
---
config:
  pie:
    donutHole: 0.4
    textPosition: 0.8
---
pie showData
  title Visits by browser
  "Chrome" : 61.4
  "Safari" : 18.9
  "Edge" : 8.2
  "Firefox" : 6.1
  "Other" : 5.4
```

RENDERED OUTPUT



SYNTAX

SYNTAX	MEANING
`pie`	Declaration
`pie showData`	Show values in legend
`title Text`	Chart title
`"Label" : 42`	Slice
`"Label" : 12.75`	Decimal value

PIE: CONFIG

KEY	EFFECT
`textPosition: 0.75`	Label position, 0 center → 1 edge
`donutHole: 0.4`	Donut, 0 – 0.9 (v11.16)
`legendPosition: bottom`	`top` `bottom` `left` `right` `center`
`highlightSlice: Safari`	Emphasize one slice (or `hover`)

THEME VARIABLES

VARIABLE	CONTROLS
`pie1 … pie12`	Slice colors
`pieTitleTextSize`	Title size
`pieSectionTextColor`	Label color
`pieStrokeColor`	Slice borders
`pieOuterStrokeWidth`	Outer ring
`pieOpacity`	Slice opacity

⚠ COMMON MISTAKES

- Leaving labels unquoted (`Chrome : 61`). Pie labels must be in double quotes.
- Using negative numbers or zero for a slice; pie values must be positive.
- Using a pie for more than about six slices or for data that isn't parts of one whole. Use `xychart` bars (sheet 31).

✓ TIPS & BEST PRACTICES

- Order slices largest first so the chart reads clockwise from the biggest share.
- Group small values into an "Other" slice.
- Add `showData` when exact numbers matter; the legend then shows each value.

QUICK SUMMARY

- `pie` / `pie showData` — declaration
- `"Label" : value` — one slice
- `donutHole` — donut chart
- `pie1…pie12` — slice colors

Quadrant Chart

TWO AXES, FOUR BOXES

A `quadrantChart` plots items on two axes and splits the space into four labeled **quadrants**, like an effort/impact or urgent/important matrix. Name each axis with a low and high label (`x-axis Low --> High`), label the quadrants, then list **points** as `Name: [x, y]` with coordinates from 0 to 1. Points can be styled inline or with `classDef`.

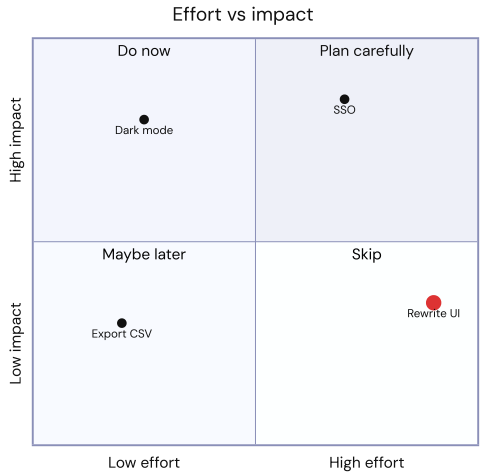
NOTES

- Coordinates run from `0` to `1` on both axes
- `quadrant-1` is top right, then counter-clockwise
- Axis labels: `x-axis Left --> Right` ; right part optional
- Style order: inline style, then class, then theme

FEATURE PRIORITIES

```
quadrantChart
  title Effort vs impact
  x-axis Low effort --> High effort
  y-axis Low impact --> High impact
  quadrant-1 Plan carefully
  quadrant-2 Do now
  quadrant-3 Maybe later
  quadrant-4 Skip
  Dark mode: [0.25, 0.8]
  SSO: [0.7, 0.85]
  Export CSV: [0.2, 0.3]
  Rewrite UI: [0.9, 0.35] radius: 8, color: #d33
```

RENDERED OUTPUT



SYNTAX

SYNTAX	MEANING
`quadrantChart`	Declaration
`title Text`	Title
`x-axis Low --> High`	X labels
`y-axis Low --> High`	Y labels
`quadrant-1 Text`	Top right
`quadrant-2 Text`	Top left
`quadrant-3 Text`	Bottom left
`quadrant-4 Text`	Bottom right
`Name: [0.3, 0.6]`	Point

POINT STYLING

SYNTAX	EFFECT
`A: [x, y] radius: 10`	Point size
`A: [x, y] color: #f30`	Fill color
`stroke-color: #000`	Border color
`stroke-width: 2px`	Border width
`A:::hot: [x, y]`	Apply a class
`classDef hot color: #f30`	Define a class

THEME VARIABLES

VARIABLE	CONTROLS
`quadrant1Fill … 4Fill`	Quadrant backgrounds
`quadrant1TextFill …`	Quadrant text
`quadrantPointFill`	Default point color
`quadrantXAxisTextFill`	X-axis text
`quadrantTitleFill`	Title

⚠ COMMON MISTAKES

- Using raw scores (`[30, 80]`). Coordinates must be scaled to 0–1.
- Assuming `quadrant-1` is top left. It's top right, then counter-clockwise like the math convention.
- Writing the class before the colon in the wrong place: it's `Name:::class: [x, y]` .

✓ TIPS & BEST PRACTICES

- Put the "good" direction up and right so quadrant-1 means "best" or "most attention".
- Keep point names short; long names overlap in dense areas.
- Use one highlight color for the items being discussed and leave the rest default.

QUICK SUMMARY

- `quadrantChart` — declaration
- `x-axis` / `y-axis Low --> High` — axes
- `quadrant-1…4` — labels, starting top right
- `Name: [x, y]` — points from 0 to 1

XY Chart

BARS AND LINES

An `xychart` draws **bar** and **line** series against an x-axis and a y-axis. The x-axis is usually a list of categories in square brackets; it can also be a numeric range. The y-axis is always numeric: give it a title and an optional `min --> max`. Each `bar [ … ]` or `line [ … ]` adds a series with one value per category; name a series to put it in the legend (v11.17). `xychart horizontal` turns the chart sideways.

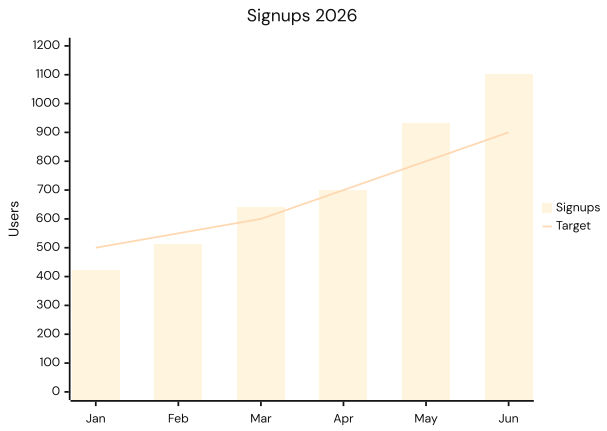
NOTES

- Text with spaces needs quotes: `"Revenue ($)"`
- No y range → Mermaid picks one from the data
- `xychart-beta` still works as an alias
- `showDataLabel: true` prints values on bars (v11.14)

MONTHLY SIGNUPS

```
xychart
  title "Signups 2026"
  x-axis [Jan, Feb, Mar, Apr, May, Jun]
  y-axis "Users" 0 --> 1200
  bar "Signups" [420, 510, 640, 700, 930, 1100]
  line "Target" [500, 550, 600, 700, 800, 900]
```

RENDERED OUTPUT



SYNTAX

SYNTAX	MEANING
`xychart`	Vertical chart
`xychart horizontal`	Horizontal chart
`title "Text"`	Title
`x-axis [a, b, c]`	Category axis
`x-axis "Year" 2020 --> 2026`	Numeric x range
`y-axis "Units" 0 --> 100`	Y title and range
`bar [1, 2, 3]`	Bar series
`line [1, 2, 3]`	Line series
`bar "Name" [ … ]`	Named → legend

XYCHART: CONFIG

KEY	EFFECT
`width / height`	Chart size
`showDataLabel: true`	Values on bars
`chartOrientation`	`vertical` / `horizontal`
`xAxis.labelFontSize`	Axis label size
`showTitle: false`	Hide the title

THEME VARIABLES

XYCHART.	CONTROLS
`plotColorPalette`	Series colors, comma list
`backgroundColor`	Background
`titleColor`	Title
`xAxisLineColor`	X-axis line
`yAxisLabelColor`	Y-axis labels

⚠ COMMON MISTAKES

- Giving a series a different number of values than there are categories. Extra or missing values misalign the chart.
- Forgetting quotes around titles with spaces: `y-axis Revenue in $` fails; use `"Revenue in $"`.
- Trying categories on the y-axis. Only the x-axis can be categorical.

✓ TIPS & BEST PRACTICES

- Set the y range explicitly (`0 --> …`) so bars aren't exaggerated by an auto range that doesn't start at 0.
- Name series when there are two or more, so the legend explains them.
- Change colors with `themeVariables: xyChart: plotColorPalette: "#4e79a7, #f28e2b"`.

QUICK SUMMARY

- `xychart` (`horizontal`) — declaration
- `x-axis [cats]` / `y-axis min --> max` — axes
- `bar [ … ]` / `line [ … ]` — series
- `"Name"` before values — legend entry

Sankey

FLOWS BETWEEN STAGES

A `sankey` diagram shows quantities flowing from sources to targets; link width is proportional to the value. The body is plain **CSV** with exactly three columns: `source,target,value`. A node that appears as both a target and a source becomes a middle stage. Empty lines are allowed for readability, and names with commas or quotes follow CSV quoting rules.

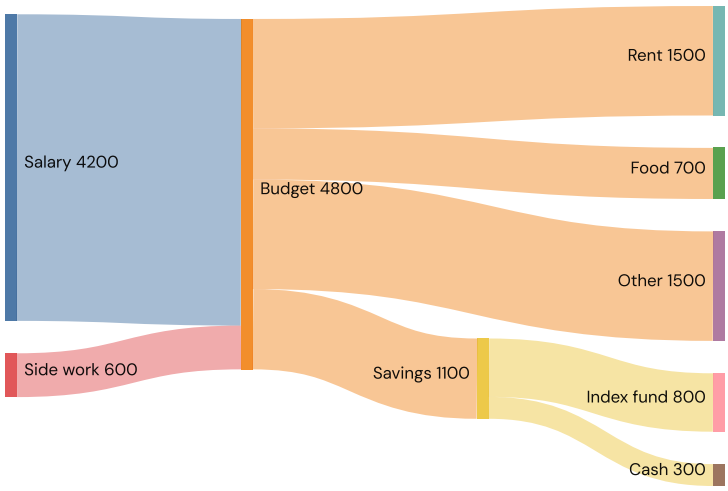
NOTES

- Exactly 3 columns per line: source, target, value
- Commas inside a name: wrap it in `"…"`
- Quote inside a quoted name: double it `""`
- `sankey-beta` still works as an alias

MONTHLY BUDGET

```
---
config:
  sankey:
    linkColor: source
    nodeAlignment: justify
---
sankey
Salary,Budget,4200
Side work,Budget,600
Budget,Rent,1500
Budget,Food,700
Budget,Savings,1100
Budget,Other,1500
Savings,Index fund,800
Savings,Cash,300
```

RENDERED OUTPUT



SYNTAX

LINE	MEANING
`sankey`	Declaration
`A,B,10`	Flow of 10 from A to B
`"A, Inc",B,5`	Name with a comma
`"Say ""hi""",B,1`	Name with quotes
`(empty line)`	Allowed, ignored
`%% comment`	Comment line

SANKEY: CONFIG

KEY	VALUES
`linkColor`	`source` `target` `gradient` or #hex
`nodeAlignment`	`justify` `center` `left` `right`
`showValues: false`	Hide numbers on nodes
`labelStyle: outlined`	Readable labels (v11.15)
`nodeWidth / nodePadding`	Node size, spacing (v11.15)
`nodeColors: { A: "#hex" }`	Fixed node colors (v11.15)
`width / height`	Diagram size

GOOD USES

USE	FLOW
Budgets	Income → categories
Energy	Source → use → loss
Funnels	Visit → signup → paid
Traffic	Referrer → page → exit
Migrations	Old system → new system

⚠ COMMON MISTAKES

- Adding a header row (`source,target,value`). It's read as data; comment it out with `%%` .
- Using spaces after commas (`A, B, 10`). They become part of the names, creating separate nodes.
- Making a loop (A → B → A). Sankey flows must not form cycles; the diagram fails to render.

✓ TIPS & BEST PRACTICES

- Export the CSV straight from a spreadsheet and paste it under `sankey` .
- Use `linkColor: gradient` when you want to trace flows across several stages.
- Round values; the picture is about proportions, not decimals.

QUICK SUMMARY

- `sankey` — declaration
- `source,target,value` — one CSV row per flow
- CSV quoting — names with commas or quotes
- `linkColor` / `nodeAlignment` — main config

Radar BETA

SCORES ON SEVERAL AXES

A `radar-beta` chart (also called spider chart) compares several items across the same set of **axes** arranged in a circle. Declare axes with `axis id["Label"]`, then add one **curve** per item with a list of values in axis order, or as `axis: value` pairs. Options set the scale (`max`, `min`), the grid shape (`graticule circle` or `polygon`) and the number of rings (`ticks`).

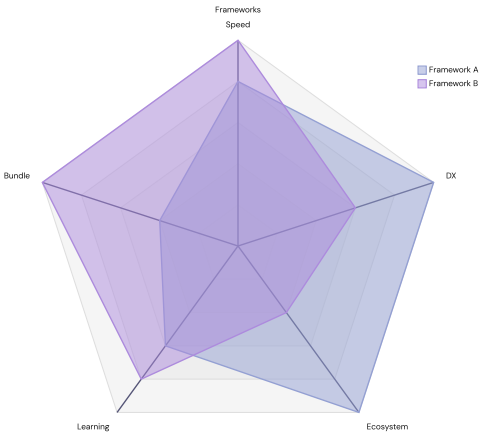
NOTES

- Several axes or curves can share a line, comma-separated
- Values follow axis order unless written as `id: value`
- Without `max`, the largest data value sets the scale
- Curve colors come from `cScale0`, `cScale1`, …
- Added in v11.6

FRAMEWORK COMPARISON

```
radar-beta
  title Frameworks
  axis perf["Speed"], dx["DX"], eco["Ecosystem"]
  axis learn["Learning"], size["Bundle"]
  curve a["Framework A"]{4, 5, 5, 3, 2}
  curve b["Framework B"]{5, 3, 2, 4, 5}
  max 5
  min 0
  graticule polygon
  ticks 5
```

RENDERED OUTPUT



SYNTAX

SYNTAX	MEANING
`radar-beta`	Declaration
`title Text`	Title
`axis a, b, c`	Axes (ids as labels)
`axis a["Speed"]`	Axis with label
`curve x["Name"]{1, 2, 3}`	Values in axis order
`curve y{ c: 3, a: 1, b: 2 }`	Values by axis id

OPTIONS

OPTION	EFFECT
`max 100`	Outer ring value
`min 0`	Center value (default 0)
`graticule circle`	Round grid (default)
`graticule polygon`	Straight-edged grid
`ticks 5`	Number of rings
`showLegend false`	Hide legend

STYLING

SETTING	CONTROLS
`cScale0 … cScale11`	Curve colors
`radar.curveOpacity`	Fill opacity (0.7)
`radar.axisColor`	Axis lines
`radar.graticuleColor`	Grid color
`curveTension (config)`	Curve rounding

⚠ COMMON MISTAKES

- Giving a curve fewer values than there are axes. The missing axes get no point; use the `id: value` form if data is partial.
- Comparing values on different scales (price in $ next to a 1–5 score). Normalize everything to one scale first.
- Plotting more than about four curves; overlapping fills become unreadable.

✓ TIPS & BEST PRACTICES

- Make "higher is better" true for every axis (invert cost-type metrics) so a bigger shape always means better.
- Set `max` explicitly so two charts can be compared side by side.
- Use `graticule polygon` when there are few axes; it reads more like a grid.

QUICK SUMMARY

- `radar-beta` — declaration
- `axis id["Label"]` — axes
- `curve id["Name"]{…}` — one series
- `max` · `min` · `graticule` · `ticks` — scale and grid

Treemap BETA

HIERARCHY SIZED BY VALUE

A `treemap-beta` shows a hierarchy as nested rectangles whose **area is proportional to value**. Like a mindmap, **indentation** creates the hierarchy. Section (parent) nodes are just a quoted name; leaf nodes are a quoted name, a colon and a number. Parents are sized by the total of their leaves. Style nodes with `:::class` and `classDef`, and format numbers with `valueFormat`.

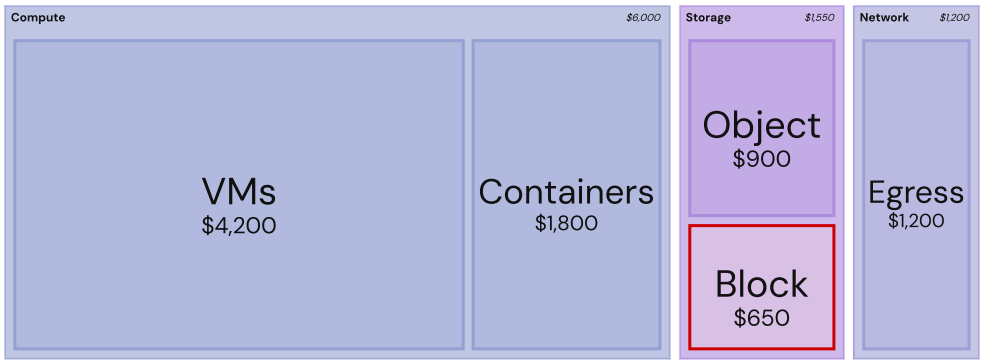
NOTES

- All names in double quotes
- Only leaves get values; sections sum their children
- Indent with spaces or tabs — but consistently
- Syntax may still change (beta)

CLOUD BILL

```
---
config:
  treemap:
    valueFormat: "$,"
---
treemap-beta
"Compute"
  "VMs": 4200
  "Containers": 1800
"Storage"
  "Object": 900
  "Block": 650:::hot
"Network"
  "Egress": 1200
classDef hot fill:#f8d7da,stroke:#c00
```

RENDERED OUTPUT



SYNTAX

SYNTAX	MEANING
`treemap-beta`	Declaration
`"Section"`	Parent node
`"Leaf": 25`	Leaf with value (indented)
`"Section":::cls`	Class on a section
`"Leaf": 25:::cls`	Class on a leaf
`classDef cls fill:#fdd`	Define a class

TREEMAP: CONFIG

KEY	EFFECT
`valueFormat`	d3 number format
`showValues: false`	Hide numbers
`padding: 10`	Gap between nodes
`diagramPadding: 8`	Outer padding
`labelFontSize: 14`	Label size
`valueFontSize: 12`	Value size

VALUEFORMAT

FORMAT	4200 SHOWS
`,`	4,200 (default)
`$`	$4200
`$,`	$4,200
`$,.2f`	$4,200.00
`.1%`	Percent (for 0–1 data)

⚠ COMMON MISTAKES

- Giving a section its own value. Only leaves carry numbers; a section's size is the sum of its children.
- Forgetting quotes around names. Unquoted names fail to parse.
- Using a treemap for flat data with no hierarchy — a bar chart is easier to read.

✓ TIPS & BEST PRACTICES

- Use treemaps for "where does it go" questions: disk usage, cloud costs, bundle size.
- Limit depth to two or three levels so labels still fit inside rectangles.
- Highlight the one or two items under discussion with a class; leave the rest themed.

QUICK SUMMARY

- `treemap-beta` — declaration
- indentation — hierarchy
- `"Leaf": value` — sized leaves
- `:::class` / `valueFormat` — styling and numbers

Venn BETA

OVERLAPPING SETS

A `venn-beta` diagram draws **sets** as overlapping circles. Declare each set with `set`, then name overlaps with `union A,B["Label"]`; unions can join three or more sets and Mermaid fills in the implied pairwise overlaps. Indented `text` lines put items inside the most recent set or union, and `:N` suffixes change relative sizes. It's one of the v12 types that defaults to `redux-color` and `neo`.

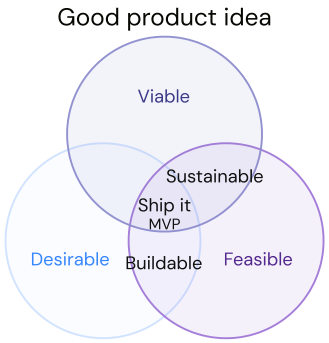
NOTES

- Sets in a `union` must be declared earlier with `set`
- Ids: bare words or quoted strings
- `["…"]` sets the display label, keeping a short id
- Added in v11.12.3; syntax may change

PRODUCT FIT

```
venn-beta
  title Good product idea
  set D["Desirable"]
  set F["Feasible"]
  set V["Viable"]
  union D,F["Buildable"]
  union F,V["Sustainable"]
  union D,F,V["Ship it"]
    text s1["MVP"]
  style D fill:#cfe2ff
```

RENDERED OUTPUT



SYNTAX

SYNTAX	MEANING
`venn-beta`	Declaration
`title "Text"`	Title
`set A`	Set
`set A["Alpha"]`	Set with label
`set A:20`	Relative size
`union A,B["AB"]`	Overlap of two sets
`union A,B,C`	Overlap of three
`text t1["Item"]`	Item in last set/union

STYLING

SYNTAX	EFFECT
`style A fill:#fdd`	Fill color
`style A,B color:#333`	Text color, several ids
`style A stroke:#900`	Outline color
`style A stroke-width:2px`	Outline width
`style A fill-opacity:0.4`	Transparency
`style t1 color:red`	Style a text item

GOOD USES

USE	SETS
`Skills overlap`	Frontend · Backend · Ops
`Feature fit`	Desirable · Feasible · Viable
`Audience`	Customers · Users · Buyers
`Tool choice`	Fast · Cheap · Good

⚠ COMMON MISTAKES

- Using a set in `union` before declaring it with `set`. Every id in a union must already exist.
- Expecting exact areas. Sizes are relative hints; the layout still has to fit circles together.
- Going beyond three sets. Four-set Venns are hard to draw and harder to read; consider a table.

✓ TIPS & BEST PRACTICES

- Keep set ids to one letter and put words in the labels: `set D["Desirable"]`.
- Use `text` items for examples so readers see what lives in each region.
- Lower `fill-opacity` if overlaps look muddy.

QUICK SUMMARY

- `venn-beta` — declaration
- `set A["Label"]` — circles
- `union A,B["Label"]` — overlaps
- `text` / `style` — items and colors

C4 Diagram

C4Context · C4Container · C4Component · Person / System / Container · Boundary · Rel

SOFTWARE ARCHITECTURE IN LEVELS

Mermaid's **C4** diagrams follow Simon Brown's C4 model and use the **C4-PlantUML** syntax: function-style statements such as `Person(alias, "Label", "Description")`. Pick the level with the declaration — `C4Context`, `C4Container`, `C4Component`, `C4Dynamic` or `C4Deployment` — then add elements, nest them in boundaries `{ }`, and connect them with `Rel(from, to, "label")`. Layout follows statement order, not an automatic algorithm.

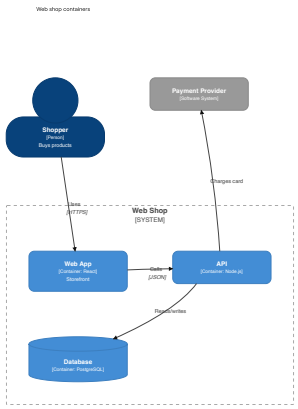
NOTES

- Still marked experimental in the docs
- `_Ext` versions draw external (grey) elements
- Optional params by name: `$techn="…"`, `$link="…"`
- Text wraps by default since v11.17.1
- Reorder statements to move shapes

SHOP — CONTAINER LEVEL

```
C4Container
  title Web shop containers
  Person(user, "Shopper", "Buys products")
  System_Boundary(shop, "Web Shop") {
    Container(web, "Web App", "React", "Storefront")
    Container(api, "API", "Node.js")
    ContainerDb(db, "Database", "PostgreSQL")
  }
  System_Ext(pay, "Payment Provider")
  Rel(user, web, "Uses", "HTTPS")
  Rel(web, api, "Calls", "JSON")
  Rel(api, db, "Reads/writes")
  Rel(api, pay, "Charges card")
```

RENDERED OUTPUT



LEVELS

DECLARATION	SHOWS
`C4Context`	System and its users
`C4Container`	Apps and data stores inside
`C4Component`	Parts of one container
`C4Dynamic`	Numbered interactions
`C4Deployment`	Nodes / infrastructure

ELEMENTS

STATEMENT	DRAWS
`Person(id, "L", "Desc")`	Person
`System(id, "Label")`	Software system
`SystemDb / SystemQueue`	Database / queue system
`Container(id, "L", "Tech")`	Container
`ContainerDb(…)`	Data store
`Component(id, "L", "Tech")`	Component
`System_Ext(…)`	External element
`Deployment_Node(…)`	Infrastructure node

BOUNDARIES & RELATIONS

STATEMENT	MEANING
`System_Boundary(id, "L")`	Group in a system, { }
`Enterprise_Boundary(…)`	Organization boundary
`Rel(a, b, "label", "tech")`	Relationship
`BiRel(a, b, …)`	Two-way relationship
`Rel_U / _D / _L / _R`	Hint direction
`UpdateElementStyle(…)`	Recolor an element
`UpdateLayoutConfig(…)`	Shapes / boundaries per row

⚠ COMMON MISTAKES

- Expecting automatic layout. C4 places shapes in statement order; reorder lines or use `UpdateLayoutConfig` to fix crowding.
- Mixing levels in one diagram (people, containers and classes). Draw one diagram per level.
- Using unsupported C4-PlantUML features such as tags, sprites or `Lay_*` statements — they aren't implemented.

✓ TIPS & BEST PRACTICES

- Start with `C4Context` for non-technical readers, then zoom into `C4Container` for engineers.
- Always put the technology in `Rel` and `Container` ("HTTPS", "PostgreSQL"); it's what reviewers ask about.
- For freer layouts consider `architecture-beta` (sheet 37) or a flowchart.

QUICK SUMMARY

- `C4Context` … `C4Deployment` — pick the level
- `Person` · `System` · `Container` · `Component` — elements
- `*_Boundary(…) { }` — grouping
- `Rel(a, b, "label")` — relationships

Architecture BETA

CLOUD SERVICES AND CONNECTIONS

An `architecture-beta` diagram shows **services** (nodes with icons), **groups** that contain them, and **edges** between them. Every element has an id, an icon in parentheses and a label in brackets: `service db(database)[Orders DB] in api`. Edges name the **side** each end attaches to — T, B, L, R — which is how you steer the layout. Five icons are built in; register an Iconify pack for thousands more.

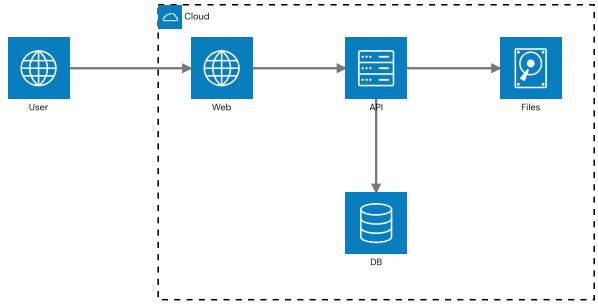
NOTES

- Built-in icons: `cloud` `database` `disk` `internet` `server`
- Other icons: `pack:name` after `registerIconPacks()`
- `in groupId` puts a service or group inside a group
- `junction` splits one edge four ways
- `align row` / `align column` line up siblings (v11.16)

SMALL WEB APP

```
architecture-beta
  group cloud(cloud)[Cloud]
  service web(internet)[Web] in cloud
  service api(server)[API] in cloud
  service db(database)[DB] in cloud
  service files(disk)[Files] in cloud
  service user(internet)[User]
  user:R --> L:web
  web:R --> L:api
  api:B --> T:db
  api:R --> L:files
```

RENDERED OUTPUT



ELEMENTS

SYNTAX	MEANING
`group g(cloud)[Title]`	Group with icon
`group g2(cloud)[T] in g`	Nested group
`service s(server)[Label]`	Service
`service s(server)[L] in g`	Service in a group
`junction j`	4-way split point
`service s(logos:aws-s3)[S3]`	Icon from a pack

EDGES

SYNTAX	MEANING
`a:R -- L:b`	Right of a to left of b
`a:T -- B:b`	Top of a to bottom of b
`a:R --> L:b`	Arrow into b
`a:R <--> L:b`	Arrows both ends
`a{group}:B --> T:b{group}`	Edge between groups
`align row a b c`	Same row (v11.16)
`align column a b`	Same column

ARCHITECTURE: CONFIG

KEY	EFFECT
`iconSize: 80`	Service icon size
`fontSize: 16`	Label size
`idealEdgeLengthMultiplier`	Spacing within groups
`nodeSeparation: 75`	Sibling spacing
`seed: 1`	Same layout every render

⚠ COMMON MISTAKES

- Referencing an id before it's declared. Groups and services must be declared before edges use them.
- Leaving out the sides (`api --> db`). Architecture edges need `:T|B|L|R` on both ends.
- Using a `pack:icon` name without registering the pack. The service shows a fallback icon.

✓ TIPS & BEST PRACTICES

- Point edges the way you read: left-to-right for request flow, top-to-bottom for data stores.
- Register Iconify packs (`logos` , `mdi`) in `mermaid.registerIconPacks()` to get vendor icons like AWS or Azure.
- If two siblings overlap, add `align row` or `align column` instead of fiddling with spacing.

QUICK SUMMARY

- `architecture-beta` — declaration
- `group` / `service` / `junction` — elements with `(icon)[label]`
- `a:R --> L:b` — edges name the sides
- `in g` — place inside a group

Block Diagram

BLOCKS YOU PLACE YOURSELF

A `block` diagram lets **you** control placement instead of a layout engine. Set the number of `columns`; blocks then fill the grid left to right, top to bottom, in the order written. A block can **span** several columns (`id:2`), `space` leaves an empty cell, and `block:id … end` nests a group with its own columns. Shapes reuse flowchart brackets, and links (`-->`) are drawn on top of the grid.

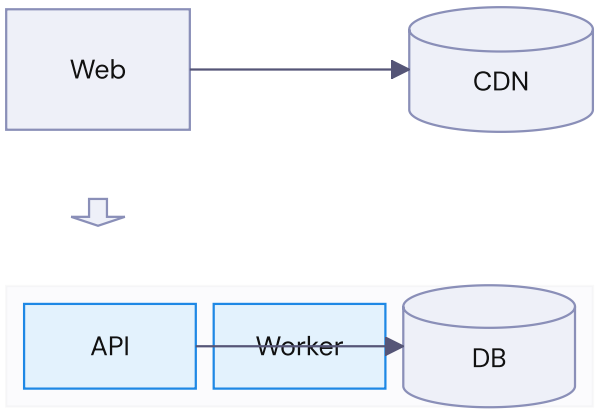
NOTES

- Several blocks can share a line: `a b c`
- Without `columns`, blocks sit in one row
- Arrow blocks: `id<["text"]>(right)` — `left up down x y`
- `block-beta` still works as an alias

THREE-TIER APP

```
block
  columns 3
  web["Web"] space:1 cdn[("CDN")]
  arrow<[" "]>(down) space:2
  block:back:3
    api["API"] worker["Worker"] db[("DB")]
  end
  web --> cdn
  api --> db
  classDef tier fill:#e3f2fd,stroke:#1e88e5
  class api,worker tier
```

RENDERED OUTPUT



LAYOUT

SYNTAX	MEANING
`block`	Declaration
`columns 3`	Grid width
`a b c`	Blocks in order
`a:2`	Span 2 columns
`space`	Empty cell
`space:2`	Two empty cells
`block:g:2 … end`	Nested group spanning 2
`columns auto`	Group sizes itself

SHAPES

SYNTAX	SHAPE
`a["Label"]`	Rectangle
`a("Label")`	Rounded
`a(["Label"])`	Stadium
`a[("Label")]`	Cylinder
`a(("Label"))`	Circle
`a{"Label"}`	Diamond
`a{{"Label"}}`	Hexagon
`a<["Label"]>(right)`	Arrow block

LINKS & STYLE

SYNTAX	MEANING
`a --> b`	Arrow
`a --- b`	Line
`a -- "text" --> b`	Labeled link
`style a fill:#fdd`	Style one block
`classDef c fill:#eef`	Define a class
`class a,b c`	Apply class

⚠ COMMON MISTAKES

- Expecting blocks to move to make links neat. Placement is fixed by the grid; links just draw over it.
- Spanning more columns than exist (`a:4` with `columns 3`). The block overflows onto the next row.
- Forgetting `end` after a nested `block:` group; the rest of the diagram ends up inside it.

✓ TIPS & BEST PRACTICES

- Sketch the grid on paper first; count cells, then write rows as lines of source.
- Use `space` cells to line up blocks across rows instead of padding labels.
- Use block diagrams for layered stacks (UI / API / DB) where position carries meaning.

QUICK SUMMARY

- `block` + `columns n` — grid
- `id:n` / `space` — span and gaps
- `block:id … end` — nested groups
- `a --> b` — links over the grid

Packet

BITS IN A HEADER

A `packet` diagram shows the layout of a network packet, file header or hardware register as rows of **bit fields**. Each line gives a bit range and a quoted label: `0-15: "Source Port"`, or a single bit `106: "URG"`. Since v11.7 you can write a **bit count** instead (`+16: "Length"`), which starts where the previous field ended. Rows are 32 bits wide by default.

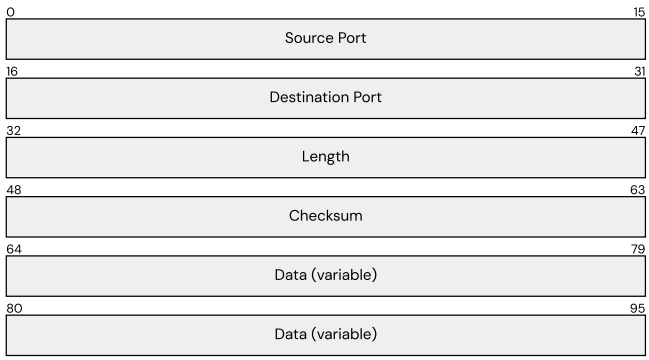
NOTES

- Ranges are inclusive: `0-15` is 16 bits
- `+n` counts from the end of the previous field
- Ranges and `+n` lines can be mixed
- Fields must not overlap or leave gaps
- `bitOrder: descending` mirrors rows for registers (v12)

UDP HEADER

```
---
config:
  packet:
    bitsPerRow: 16
    showBits: true
---
packet
  title UDP datagram
  +16: "Source Port"
  +16: "Destination Port"
  32-47: "Length"
  48-63: "Checksum"
  64-95: "Data (variable)"
```

RENDERED OUTPUT



UDP datagram

SYNTAX

SYNTAX	MEANING
`packet`	Declaration (`packet-beta` alias)
`title Text`	Title
`0-15: "Field"`	Bits 0 through 15
`16: "Flag"`	Single bit
`+8: "Field"`	Next 8 bits
`+1: "Flag"`	Next single bit

PACKET: CONFIG

KEY	EFFECT
`bitsPerRow: 16`	Row width (default 32)
`showBits: true`	Bit numbers above fields
`bitOrder: descending`	MSB left (v12)
`rowHeight: 32`	Row height
`bitWidth: 32`	Width per bit

GOOD USES

USE	EXAMPLE
`Protocol headers`	TCP, UDP, IPv4
`File formats`	PNG chunk, BMP header
`Registers`	MCU control register
`Bit flags`	Permissions, feature bits
`Wire formats`	Custom binary protocol

⚠ COMMON MISTAKES

- Counting ranges as exclusive. `0-16` is 17 bits; a 16-bit field is `0-15`.
- Leaving a gap or overlap between fields. Mermaid reports an error; every bit must belong to exactly one field.
- Writing ranges high-to-low (`7-0`) for registers. Always declare low-to-high and use `bitOrder: descending` to flip the drawing.

✓ TIPS & BEST PRACTICES

- Prefer `+n` bit counts: inserting a field later doesn't force you to renumber everything after it.
- Set `bitsPerRow: 16` or `8` for small registers so fields aren't tiny.
- Label variable-length tails clearly (`Data (variable)`).

QUICK SUMMARY

- `packet` — declaration
- `start-end: "label"` — bit range
- `+n: "label"` — bit count from the last field
- `bitsPerRow` / `bitOrder` — row width and direction

Tree View BETA

FOLDER AND FILE TREES

A `treeView-beta` diagram draws a **directory tree** with connector lines. Structure comes only from **indentation**, and a trailing `/` marks a directory (drawn in bold). You can paste an existing tree drawn with box characters (`├──`, `└──`, `│`) and Mermaid detects it automatically. Annotate entries with `## description`, `:::highlight`, or an `icon()`; file and folder icons are off unless `showIcons` is set.

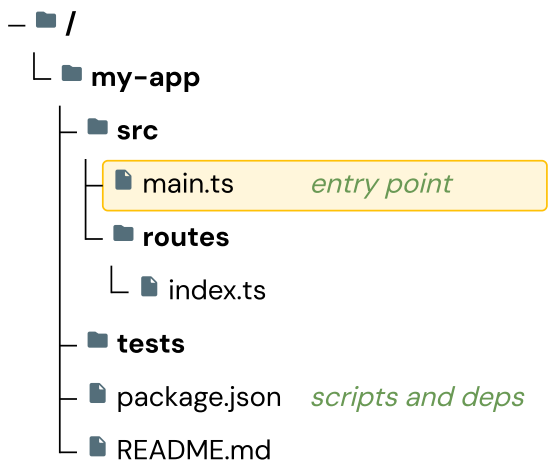
NOTES

- Quote names with spaces: `"my file.txt"`
- Tabs are expanded to spaces
- Box-drawing and indentation styles both work
- Annotations combine in any order
- Added in v11.14

PROJECT LAYOUT

```
---
config:
  treeView:
    showIcons: true
---
treeView-beta
  my-app/
    src/
      main.ts :::highlight ## entry point
      routes/
        index.ts
    tests/
    package.json ## scripts and deps
    README.md
```

RENDERED OUTPUT



SYNTAX

SYNTAX	MEANING
`treeView-beta`	Declaration
`src/`	Directory (bold)
`index.ts`	File
`"my file.txt"`	Name with spaces
`child`	Indent = child
`├── a.ts`	Box-drawing input
`%% comment`	Comment

ANNOTATIONS

SYNTAX	EFFECT
`## text`	Italic description
`:::highlight`	Built-in highlight
`:::myClass`	Custom CSS class
`icon(logos:react)`	Icon from a pack
`icon(none)`	Hide this icon

TREEVIEW: CONFIG

KEY	EFFECT
`showIcons: true`	Built-in file / folder icons
`defaultIconPack`	Pack for bare icon names
`extensionIcons`	Map `.ts` → icon
`filenameIcons`	Map `Dockerfile` → icon

⚠ COMMON MISTAKES

- Forgetting the trailing `/` on folders. An empty folder then looks like a file.
- Mixing box-drawing lines and plain indentation in one tree; pick one style.
- Expecting file-type icons out of the box. Mermaid ships no mapping; set `extensionIcons` with a registered pack.

✓ TIPS & BEST PRACTICES

- Paste the output of the `tree` command straight in — the box characters are understood.
- Use `## …` descriptions instead of a separate table explaining what each file does.
- Highlight only the files a reader should open first.

QUICK SUMMARY

- `treeView-beta` — declaration
- indentation or `├──` — structure
- trailing `/` — directory
- `##` · `:::highlight` · `icon()` — annotations

Kanban

A BOARD OF WORK ITEMS

A kanban diagram draws a board: **columns** for workflow stages and **tasks** (cards) inside them. Columns sit at the first indent level as `id[Title]`; tasks are indented under their column as `id[Description]`. Add metadata with `@{ … }` — `assigned`, `ticket` and `priority` — and set `ticketBaseUrl` in config to turn ticket numbers into links.

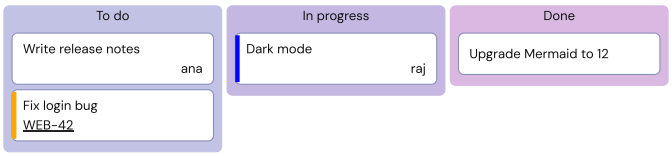
NOTES

- Indentation decides which column a task belongs to
- Ids must be unique across the board
- `priority`: `'Very High'` `'High'` `'Low'` `'Very Low'`
- `#TICKET#` in `ticketBaseUrl` is replaced by the ticket id

SPRINT BOARD

```
---
config:
  kanban:
    ticketBaseUrl: "https://jira.example.com/browse/#TICKET#"
---
kanban
  todo[To do]
    t1[Write release notes]@{ assigned: "ana" }
    t2[Fix login bug]@{ ticket: WEB-42, priority: "High" }
  doing[In progress]
    t3[Dark mode]@{ assigned: "raj", priority: "Low" }
  done[Done]
    t4[Upgrade Mermaid to 12]
```

RENDERED OUTPUT



SYNTAX

SYNTAX	MEANING
`kanban`	Declaration
`todo[To do]`	Column (first indent)
`t1[Task text]`	Task in that column
`[Task text]`	Task without an id
`t1[…]@{ … }`	Task metadata

METADATA

KEY	VALUE
`assigned: "ana"`	Person responsible
`ticket: WEB-42`	Ticket id (link)
`priority: "Very High"`	Highest
`priority: "High"`	High
`priority: "Low"`	Low
`priority: "Very Low"`	Lowest

KANBAN: CONFIG

KEY	EFFECT
`ticketBaseUrl`	Link pattern with #TICKET#
`sectionWidth`	Column width
`padding`	Space around cards

⚠ COMMON MISTAKES

- Indenting a task at the same level as columns. It becomes a new column.
- Inventing priority values (`'Medium'` , `'P1'`). Only the four listed values are recognized.
- Reusing an id in two columns. Ids must be unique, or cards collide.

✓ TIPS & BEST PRACTICES

- Use a kanban diagram for snapshots in docs or status reports, not as a live tracker.
- Set `ticketBaseUrl` once so every `ticket:` becomes a clickable link.
- Keep card text short; put details in the ticket.

QUICK SUMMARY

- `kanban` — declaration
- `id[Title]` — column at first indent
- indented `id[Text]` — task
- `@{ assigned, ticket, priority }` — metadata

Ishikawa (Fishbone) BETA

CAUSES OF A PROBLEM

An `ishikawa-beta` diagram (fishbone, cause-and-effect) puts a **problem** at the head and its **causes** on bones along the spine. The syntax is pure indentation: the first line after the declaration is the problem, the next indent level holds cause **categories**, and deeper levels hold causes and sub-causes. It's a structured way to run a root-cause discussion.

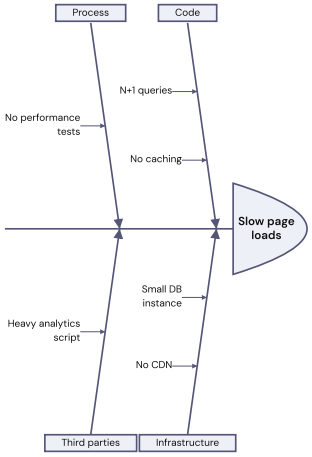
NOTES

- First line = the problem (fish head)
- Indent level 1 = categories (bones)
- Deeper levels = causes, sub-causes
- Added in v11.12.3; syntax may change

SLOW PAGE LOADS

```
ishikawa-beta
  Slow page loads
  Code
    N+1 queries
    No caching
  Infrastructure
    Small DB instance
    No CDN
  Process
    No performance tests
  Third parties
    Heavy analytics script
```

RENDERED OUTPUT



SYNTAX

LINE	MEANING
`ishikawa-beta`	Declaration
`Problem`	Head of the fish
`Category`	Main bone
`Cause`	Cause on that bone
`Sub-cause`	Deeper detail

CLASSIC 6M CATEGORIES

CATEGORY	ASK ABOUT
`Method`	Process, procedures
`Machine`	Tools, infrastructure
`Material`	Inputs, data, parts
`Measurement`	Metrics, monitoring
`Man / People`	Skills, staffing
`Mother Nature`	Environment

SOFTWARE CATEGORIES

CATEGORY	EXAMPLES
`Code`	Bugs, inefficiency
`Infrastructure`	Capacity, network
`Process`	Testing, review
`Dependencies`	Libraries, APIs
`People`	Knowledge, on-call

⚠ COMMON MISTAKES

- Writing the problem as a cause ("Because of no CDN"). The head should state the effect: "Slow page loads".
- Mixing indentation widths. A cause can jump to the wrong bone; indent two spaces per level.
- Listing solutions instead of causes; fixes belong in follow-up actions, not on the bones.

✓ TIPS & BEST PRACTICES

- Start from a standard category set (6M or the software list) so the team doesn't miss an area.
- Ask "why?" again for each cause and add the answer as a sub-cause.
- Keep each cause to a few words so the bones stay readable.

QUICK SUMMARY

- `ishikawa-beta` — declaration
- first line — the problem
- indentation — categories, causes, sub-causes
- 6M — a starting set of categories

Wardley Map BETA

VALUE CHAIN VS EVOLUTION

A `wardley-beta` map places components by **visibility** to the user (vertical, 0–1) and **evolution** from genesis to commodity (horizontal, 0–1). Coordinates use the OnlineWardleyMaps order **`[visibility, evolution]`** — y first. **Anchors** are users or needs at the top; –> links show dependencies; `evolve` draws where a component is heading; decorators like `(build)` and `(buy)` mark sourcing choices.

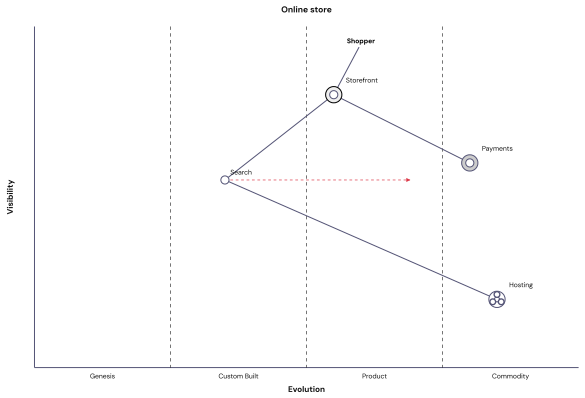
NOTES

- `[0.9, 0.2]` = high visibility, early evolution
- Names may contain spaces and hyphens without quotes
- `(inertia)` marks resistance to change
- `size [1100, 600]` sets the canvas
- Added in v11.14

ONLINE STORE

```
wardley-beta
  title Online store
  anchor Shopper [0.95, 0.60]
  component Storefront [0.80, 0.55] (build)
  component Payments [0.60, 0.80] (buy)
  component Search [0.55, 0.35]
  component Hosting [0.20, 0.85] (market)
  Shopper -> Storefront
  Storefront -> Payments
  Storefront -> Search
  Search -> Hosting
  evolve Search 0.70
```

RENDERED OUTPUT



ELEMENTS

SYNTAX	MEANING
`anchor User [0.95, 0.6]`	User or need (bold)
`component API [0.6, 0.7]`	Component
`… label [-40, 10]`	Move the label
`note "text" [0.3, 0.5]`	Note on the map
`title Text`	Title
`evolution A -> B -> C -> D`	Custom stage names

LINKS & MOVEMENT

SYNTAX	MEANING
`A -> B`	A depends on B
`A -.-> B`	Dashed flow
`A +> B`	Flow with arrow
`A +'text'> B`	Labeled flow
`evolve B 0.8`	Evolution arrow to 0.8
`pipeline B { … }`	Variants of B by evolution

DECORATORS

DECORATOR	SYMBOL / MEANING
`(build)`	Triangle — build in-house
`(buy)`	Diamond — buy product
`(outsource)`	Square — outsource
`(market)`	Circle — utility / market
`(inertia)`	Bar — resists change

⚠ COMMON MISTAKES

- Writing coordinates as `[x, y]`. Wardley uses `[visibility, evolution]` — vertical first.
- Putting infrastructure near the top. Low visibility (bottom) means further from the user.
- Using `evolve` with a value lower than the current evolution; components move toward commodity.

✓ TIPS & BEST PRACTICES

- Start from the anchor (user need) and add only components the user's need depends on.
- Use `(build)` only where you differentiate; most commodity parts should be `(buy)` or `(market)`.
- Draw the map twice — today and in 18 months — to drive strategy talks.

QUICK SUMMARY

- `wardley-beta` — declaration
- `[visibility, evolution]` — coordinates, y first
- `anchor` / `component` — needs and parts
- `->` · `evolve` · decorators — dependencies, movement, sourcing

Cynefin BETA

WHAT KIND OF PROBLEM IS IT?

A `cynefin-beta` diagram draws Dave Snowden's **Cynefin** sense-making framework: five domains that call for different responses. **Clear** (best practice), **Complicated** (analyze, good practice), **Complex** (probe, emergent practice), **Chaotic** (act first) and **Confusion** in the middle. Open a domain with its keyword and list quoted items under it; `-->` lines show items moving between domains.

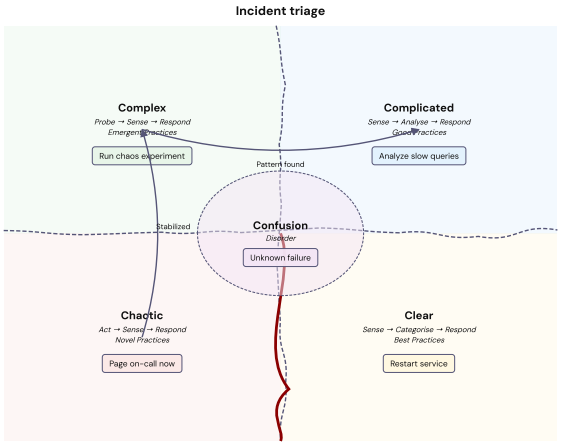
NOTES

- Items are quoted strings, one per line
- Confusion shows up to 3 items, then `+N more`
- The "cliff" sits between Clear and Chaotic
- Empty domains still render — handy as a worksheet
- Added in v11.16

INCIDENT TRIAGE

```
cynefin-beta
  title Incident triage
  clear
    "Restart service"
  complicated
    "Analyze slow queries"
  complex
    "Run chaos experiment"
  chaotic
    "Page on-call now"
  confusion
    "Unknown failure"
  chaotic --> complex : "Stabilized"
  complex --> complicated : "Pattern found"
```

RENDERED OUTPUT



DOMAINS

KEYWORD	RESPONSE
`clear`	Sense → categorize → respond
`complicated`	Sense → analyze → respond
`complex`	Probe → sense → respond
`chaotic`	Act → sense → respond
`confusion`	Find out which domain

SYNTAX

SYNTAX	MEANING
`cynefin-beta`	Declaration
`title Text`	Title
`complex`	Open a domain block
`"Item"`	Item in that domain
`a --> b : "label"`	Transition

COMMON TRANSITIONS

MOVE	MEANS
`complex --> complicated`	Pattern understood
`complicated --> clear`	Practice codified
`chaotic --> complex`	Crisis stabilized
`clear --> chaotic`	Complacency (cliff)

⚠ COMMON MISTAKES

- Leaving items unquoted. Each item must be a quoted string on its own line.
- Putting long lists in one domain. Quadrants have a fixed size, and long lists overflow.
- Treating Confusion as a fifth quadrant; it's the state of not knowing, and items should move out of it.

✓ TIPS & BEST PRACTICES

- Use an empty Cynefin diagram as a workshop template, then fill it in together.
- Write items as situations ("Unknown failure"), not actions, so the domain decides the response.
- Add transitions to show how a problem's domain changed over an incident.

QUICK SUMMARY

- `cynefin-beta` — declaration
- `clear` · `complicated` · `complex` · `chaotic` · `confusion` — domains
- `"Item"` — items under a domain
- `a --> b : "label"` — transitions

Quick Reference — Core Syntax

skeleton · flowchart · sequence · ER · state · class · gantt · git

DIAGRAM SKELETON

SYNTAX	DOES	#
`--- title: Text ---`	front matter + title	02
`config: theme: dark`	per-diagram config	21
`flowchart LR`	declaration line	02
`%% text`	comment	02
`accTitle: · accDescr:`	accessible title / desc	02
`id["A (b) & c"]`	quote special chars	02
`#quot; · #35;`	character by code	02
`"`**b** _i_`"`	markdown string, wraps	02
`Line 1 Line 2`	line break in a label	02

FLOWCHART SHAPES

SYNTAX	DOES	#
`flowchart TD · LR · BT · RL`	direction	05
`A[Text]`	process box	05
`A(Text)`	rounded box	05
`A([Text])`	start / end	05
`A{Text}`	decision	05
`A[(Text)]`	database	05
`A((Text))`	circle	05
`A[/Text/]`	input / output	05
`A@{ shape: doc }`	named shape (v11.3)	05

FLOWCHART LINKS

SYNTAX	DOES	#
`A --> B`	arrow	06
`A --- B`	line, no head	06
`A -.-> B · A ==> B`	dotted · thick	06
`A -->|text| B`	labeled link	06
`A & B --> C`	fan in / out	06
`A ~~~ B`	invisible (layout)	06
`subgraph id [Title]`	group … end	06
`click A "url" "tip"`	clickable node	06
`linkStyle 0 stroke:red`	style link #0	06

SEQUENCE

SYNTAX	DOES	#
`participant A as Alice`	box, short id	07
`actor A`	stick figure	07
`A->>B: msg`	call (solid)	07
`B-->>A: msg`	reply (dotted)	07
`A-)B · A-xB`	async · lost	07
`A->>+B · B-->>-A`	(de)activate	08
`Note over A,B: …`	note over both	08
`loop · alt / else · opt · par`	blocks … end	08
`autonumber`	numbering	07

ENTITY RELATIONSHIP

SYNTAX	DOES	#
`erDiagram`	declaration	09
`USER { int id PK }`	attribute + key	09
`string email UK "note"`	unique + comment	09
`p[Person]`	alias label	09
`A ||--o{ B : "has"`	one-to-many	10
`A ||--|| B`	one-to-one	10
`A }o--o{ B`	many-to-many	10
`|o · || · }o · }|`	0..1 · 1 · 0+ · 1+	10
`-- · ..`	solid · dashed	10

STATE

SYNTAX	DOES	#
`stateDiagram-v2`	declaration	11
`[*] --> A · A --> [*]`	start · end	11
`A --> B : event`	on event	11
`state "Long name" as s1`	display name	11
`note right of A : text`	one-line note	11
`state X { … }`	composite state	12
`--`	parallel regions	12
`state c <<choice>>`	choice	12
`<<fork>> · <<join>>`	split · merge	12

CLASS

SYNTAX	DOES	#
`class A { +int x }`	class + member	13
`+ · - · # · ~`	pub · priv · prot · pkg	13
`draw()* · count$`	abstract · static	13
`class Box~T~`	generic	13
`<<interface>>`	annotation	13
`A <|-- B`	B extends A	14
`A *-- B · A o-- B`	owns · has	14
`A ..> B · A ..|> B`	uses · implements	14
`A "1" --> "*" B`	cardinality	14

GANTT & TIMELINE

SYNTAX	DOES	#
`dateFormat YYYY-MM-DD`	input format	15
`axisFormat %b %d`	axis labels	15
`section Name`	group of rows	15
`Task :a1, 2026-03-02, 5d`	id · start · span	15
`Task :after a1, 3d`	after a1	15
`done · active · crit · milestone`	task tags	15
`excludes weekends`	skip days	16
`timeline`	declaration	16
`2020 : Event : Event`	period : events	16

GIT GRAPH & MINDMAP

SYNTAX	DOES	#
`commit id: "x" tag: "v1"`	commit	19
`commit type: HIGHLIGHT`	commit style	19
`branch dev · checkout main`	create · switch	19
`merge dev`	merge into current	20
`cherry-pick id: "x"`	copy a commit	20
`mindmap · root((Idea))`	declaration · root	17
`indentation`	parent / child	17
`))Bang(( · )Cloud( · {{Hex}}`	node shapes	17
`::icon(fa fa-book) · :::cls`	icon · class	18

⚠ COMMON MISTAKES

- Lowercase `end` as node text breaks a flowchart. Write `End` or quote it.
- A node name starting with `o` or `x` right after a link (`A--oB`) turns into a circle / cross end. Add a space.
- Mixing tabs and spaces. Mindmap, treemap, tree view and Ishikawa use indentation as structure.

✓ TIPS & BEST PRACTICES

- Keep ids short and put the wording in labels: `A["Long label (x)"]` .
- Paste into `mermaid.live` to check a diagram before you commit it.
- Arrows read the same across types: `-->` solid, `-.->` / `..>` dashed.

QUICK SUMMARY

- First line declares the diagram type
- `-->` links flowchart, state and class
- Sequence: `->>` call, `-->>` reply
- `%%` comments · `---` front matter

Sheet 46

Quick Reference — Types & Config

declarations · syntax per type · themes · styling · mmdc

PROCESS & BEHAVIOR

DECLARATION	DRAWS	#
`flowchart`	steps and decisions	05
`swimlane-beta`	steps by owner	23
`sequenceDiagram`	calls between parts	07
`zenuml`	calls written as code	27
`stateDiagram-v2`	status lifecycle	11
`journey`	experience by step	24
`usecase-beta`	actors and goals	25
`requirementDiagram`	requirement tracing	26
`eventmodeling`	event-sourced design	28
`ishikawa-beta`	root causes	42

STRUCTURE & ARCHITECTURE

DECLARATION	DRAWS	#
`erDiagram`	tables and keys	09
`classDiagram`	classes, interfaces	13
`mindmap`	ideas from a topic	17
`gitGraph`	branches and merges	19
`C4Context · C4Container`	C4 model levels	36
`architecture-beta`	cloud services	37
`block`	blocks on a grid	38
`packet`	bit fields	39
`treeView-beta`	file tree	40
`wardley-beta`	value chain vs evolution	43

PLANS & CHARTS

DECLARATION	DRAWS	#
`gantt`	dated tasks	15
`timeline`	events by period	16
`kanban`	cards by column	41
`pie`	share of a whole	29
`quadrantChart`	two axes, 4 quadrants	30
`xychart`	bars / lines	31
`sankey`	flow quantities	32
`radar-beta · treemap-beta`	scores · sized tree	33
`venn-beta`	set overlap	35
`cynefin-beta`	kind of problem	44

PROCESS SYNTAX

SYNTAX	DOES	#
`subgraph Sales … end`	swimlane lane	23
`Task: 5: Me, Cat`	journey step	24
`actor U · Login("Sign in")`	use case	25
`A ..> : include B`	use case include	25
`element e { type: x }`	requirement element	26
`e - satisfies -> r`	requirement link	26
`A.method() · A->B: msg`	ZenUML sync · async	27
`tf 01 ui CartUI`	event model frame	28

CHART SYNTAX

SYNTAX	DOES	#
`"Label" : 42`	pie slice	29
`x-axis Low --> High`	quadrant axes	30
`Name: [0.3, 0.6]`	quadrant point	30
`x-axis [a, b] · bar [1, 2]`	xy axis · bars	31
`A,B,10`	sankey row	32
`axis a, b · curve x{1, 2}`	radar	33
`"Leaf": 25`	treemap leaf	34
`set A · union A,B`	venn	35

STRUCTURE SYNTAX

SYNTAX	DOES	#
`Person(id, "L") · Rel(a, b, "l")`	C4	36
`service s(server)[L] in g`	arch service	37
`a:R --> L:b`	arch edge	37
`columns 3 · a:2 · space`	block grid	38
`0-15: "Field" · +8: "F"`	packet bits	39
`todo[To do] · t1[Task]`	kanban	41
`component API [0.6, 0.7]`	wardley	43
`complex · "Item"`	cynefin	44

CONFIG & THEMES

SYNTAX	DOES	#
`config: theme: neutral`	front matter config	21
`default · neutral · dark · forest`	built-in themes	21
`base + themeVariables`	customizable theme	22
`look: classic · handDrawn · neo`	drawing style	21
`layout: dagre · elk`	layout engine	21
`flowchart: { curve: … }`	per-type settings	21
`%%{init: {…}}%%`	old directive	21
`front matter › initialize()`	which config wins	21

STYLING

SYNTAX	DOES	#
`classDef hot fill:#fdd`	define class	22
`class A,B hot`	apply to ids	22
`A:::hot`	apply inline	22
`classDef default …`	all nodes	22
`style A fill:#eef`	one element	22
`linkStyle 0,2 stroke:#999`	links by index	22
`primaryColor · lineColor`	theme vars	22
`stroke-dasharray: 5 5`	dashed	22

RENDERING & TOOLS

SYNTAX	DOES	#
```` ```mermaid ````	Markdown fence	01
`mermaid.live`	live editor	01
`mermaid.initialize({…})`	global config	03
`mermaid.run()`	render page	03
`mermaid.render(id, text)`	→ { svg }	03
`mermaid.parse(text)`	validate only	03
`mmdc -i a.mmd -o a.svg`	CLI render	03
`mmdc -i doc.md -o out.md`	render fences	03

⚠ COMMON MISTAKES

- Dropping `-beta`. Swimlane, use case, radar, treemap, Venn, architecture, tree view, Ishikawa, Wardley and Cynefin still need it.
- Named colors in `themeVariables`. Only hex values work: `#ff0000`, not `red`.
- Writing new diagrams with `%%{init}%%`. It is deprecated; use `config:` in front matter.

✓ TIPS & BEST PRACTICES

- Use `theme: neutral` (or `base` with your colors) for print.
- Try `look: handDrawn` for drafts, so readers know the design isn't final.
- Pin the Mermaid version in your CDN URL. v12 changed default looks for 10 diagram types.

QUICK SUMMARY

- Declaration picks the type: 31 types, newest end in `-beta`
- `config:` sets theme, look and layout
- `classDef` + `:::` for reusable styles
- `mmdc` for files, `mermaid.run()` for pages